

LUNA: A Model-Based Universal Analysis Framework for Large Language Models

Da Song , Graduate Student Member, IEEE, Xuan Xie , Jiayang Song , Graduate Student Member, IEEE, Derui Zhu , Yuheng Huang , Felix Juefei-Xu , Member, IEEE, and Lei Ma , Member, IEEE

Abstract—Over the past decade, Artificial Intelligence (AI) has had great success recently and is being used in a wide range of academic and industrial fields. More recently, Large Language Models (LLMs) have made rapid advancements that have propelled AI to a new level, enabling and empowering even more diverse applications and industrial domains with intelligence, particularly in areas like software engineering and natural language processing. Nevertheless, a number of emerging trustworthiness concerns and issues exhibited in LLMs, e.g., robustness and hallucination, have already recently received much attention, without properly solving which the widespread adoption of LLMs could be greatly hindered in practice. The distinctive characteristics of LLMs, such as the self-attention mechanism, extremely large neural network scale, and autoregressive generation usage contexts, differ from classic AI software based on Convolutional Neural Networks and Recurrent Neural Networks and present new challenges for quality analysis. Up to the present, it still lacks universal and systematic analysis techniques for LLMs despite the urgent industrial demand across diverse domains. Towards bridging such a gap, in this paper, we initiate an early exploratory study and propose a universal analysis framework for LLMs, named *LUNA*, which is designed to be general and extensible and enables versatile analysis of LLMs from multiple quality perspectives in a human-interpretable manner. In particular, we first leverage the data from desired trustworthiness perspectives to construct an abstract model as an auxiliary analysis asset and proxy, which is empowered by various abstract model construction methods built-in *LUNA*. To assess the quality of the abstract model, we collect and define a number of evaluation metrics, aiming at both the abstract model level and the semantics

level. Then, the semantics, which is the degree of satisfaction of the LLM w.r.t. the trustworthiness perspective, is bound to and enriches the abstract model with semantics, which enables more detailed analysis applications for diverse purposes, e.g., abnormal behavior detection. To better understand the potential usefulness of our analysis framework *LUNA*, we conduct a large-scale evaluation, the results of which demonstrate that 1) the abstract model has the potential to distinguish normal and abnormal behavior in LLM, 2) *LUNA* is effective for the real-world analysis of LLMs in practice, and the hyperparameter settings influence the performance, 3) different evaluation metrics are in different correlations with the analysis performance. In order to encourage further studies in the quality assurance of LLMs, we made all of the code and more detailed experimental results data available on the supplementary website of this paper <https://sites.google.com/view/llm-luna>.

Index Terms—Large language models, deep neural networks, model-based analysis, quality assurance.

I. INTRODUCTION

OVER the last few years, a series of tremendous performance leaps in many real-world applications across domains have been empowered by the rapid advancement of LLMs, especially in the domain of Software Engineering (SE) and Natural Language Processing (NLP), e.g., code generation [1], program repair [2], sentiment analysis [3], and question answering [4]. Representative LLM-enabled applications such as ChatGPT [5], GPT-4 [6], and Llama [7] are often recognized as the early foundation towards Artificial General Intelligence (AGI) [8]. More recently, LLMs have presented the promising potential to become new enablers and boosters to further revolutionize intelligentization and automation for various key stages of the software production life-cycle.

Despite the rapid development, the current quality [9], reliability [10], robustness [11], and explainability [12] of LLMs pose many concerns of social society and technical challenges, the research on which, on the other hand, is still at a very early stage. For example, recent research indicates that existing LLMs can occasionally generate content that is toxic, biased, insecure, or erroneous [9], [13], [14]. For example, a typically new type of quality issue is the phenomenon of hallucination [15], where LLMs confidently produce nonfactual or erroneous outputs, which poses significant challenges for their implementation, particularly in environments where safety and security

Manuscript received 22 October 2023; revised 29 May 2024; accepted 3 June 2024. Date of publication 18 June 2024; date of current version 18 July 2024. This work was supported in part by Canada CIFAR AI Chairs Program, in part by the Natural Sciences and Engineering Research Council of Canada, in part by the JST-Mirai Program under Grant JPMJMI20B8, in part by the JSPS KAKENHI under Grant JP21H04877, Grant JP23H03372, Grant JP24K02920, and in part by the Autoware Foundation. Recommended for acceptance by T. Menzies. (Da Song and Xuan Xie contributed equally to this work.) (Corresponding author: Lei Ma.)

Da Song, Xuan Xie, Jiayang Song, and Yuheng Huang are with the Department of Electrical and Computer Engineering, University of Alberta, Edmonton, T6G 1H9, Canada (e-mail: dsong4@ualberta.ca; xxie9@ualberta.ca; jiyang13@ualberta.ca; yuheng18@ualberta.ca).

Derui Zhu is with the Department of Computer Science, Technical University of Munich, D-80333 Munich, Germany (e-mail: derui.zhu@tum.de).

Felix Juefei-Xu is with New York University, New York, NY 10012, USA (e-mail: juefei.xu@nyu.edu).

Lei Ma is with The University of Tokyo, Tokyo 113-8658, Japan, and also with the University of Alberta, Canada (e-mail: ma.lei@acm.org).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TSE.2024.3411928>, provided by the authors.

Digital Object Identifier 10.1109/TSE.2024.3411928

are paramount. Moreover, the rapid industrial adoption of LLMs in various applications, e.g., robotic control [16] and medical image diagnosis [17], necessitates urgent analysis and risk assessment methodologies for LLMs.

In recent years, there has come an increasing trend in research to tackle the quality assurance challenges of deep learning software, especially Deep Neural Networks (DNNs) [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35], [36], [37], [38], [39]. Some studies have focused on DNN testing with the goal of pinpointing inputs that a DNN struggles to manage [18], [19], [20], [21], [22], [23], [24], [30], [31], [32], [33], [34], [35], [40], [41], [42], [43]. Concurrently, advancements in DNN debugging and repair [25], [26], [27], [28], [29], [36], [37], [38], [39], [44], [45], [46], [47], [48], [49] aim to understand the reasons behind a DNN's incorrect predictions and subsequently repair the model. These studies have made notable contributions to the advancement of quality assurance in DL-based software. However, a majority of these works are centered around Convolutional Neural Networks (CNNs) [50] and Recurrent Neural Networks (RNNs) [51]. Pei et al. propose DeepXplore, a white-box DNN testing framework combined with neuron coverage and differential testing to efficiently capture defects in DNN systems [18]. Zohdinasab et al. leverage illumination search to identify and quantify the dimensions of feature space in testing deep learning systems [52]. Hu et al. propose a framework for mapping between dangerous situations and the image transformations in the machine vision components [53]. Among these analysis techniques, *model-based analysis* [29], [54], [55], [56], [57], [58] has been demonstrated as an effective approach to both provide analysis results, e.g., testing and monitoring, and human-explainable results. Pan and Rajan [59] propose to decompose a CNN model into modules for each output class, enabling reusability and lowering the environmental cost. Dong et al. [60] develop an approach to extract probabilistic automata for RNN interpretation, which integrates hidden states abstraction and automata learning. Qi et al. develop ArchRepair, which repairs DNNs by jointly optimizing architecture and weights at the block level [61].

Different from CNNs and RNNs, LLMs behave with distinct features such as the adaptation of the self-attention [62] mechanism as its core, the complex and large-scale model size (e.g., 6.7 billion to 65.2 billion parameters in LLaMA series released by Meta [7]), and the generative output scheme which highly depend on a broad spectrum of user's inputs. Such features make the analysis of LLMs' behavior more challenging compared to classification contexts, which are the main focus of existing research. Therefore, even up to the present, only very limited research has been conducted to probe general-purpose LLM-oriented analysis techniques to understand the quality and behavior characteristics of LLMs from various aspects. The prospective quality assurance methods should help better comprehend LLMs' internal behaviors, identify unwanted outputs, and aid in improving the trustworthiness of LLMs in practical usage.

As described above, the philosophy of model-based analysis has been widely proven useful in providing quality assurance

for traditional DNNs; however, its effectiveness for LLMs is still unknown and deserves further investigation. Therefore, to bridge this gap, we propose and design *LUNA*, a model-based universal analysis framework for large language models. The first step of *LUNA* is to extract an assistant model for analysis. Due to the high dimensional space and sparsely distributed states, we extract and build the *abstract model* such as Discrete-Time Markov Chain (DTMC) and Hidden Markov Model (HMM), to enable and ease the analysis procedure and capture LLM's probabilistic nature. With the obtained abstract model, we further perform *semantics* binding, which is the degree of satisfaction of the LLM with respect to the desired quality perspective, to the abstract model to enable in-depth quality analysis. Moreover, to evaluate the quality of the model, we collect abstract model-wise metrics and propose semantics-wise metrics to measure from the perspectives of models and semantics, respectively. Finally, we apply the constructed abstract model on *abnormal behavior detection* to detect potentially erroneous outputs from the LLMs, e.g., hallucination. It is worth noting that while *LUNA* aims to be universally applicable across various tasks and domains, its current implementation and validation are contingent on the availability of open-source LLMs due to the requirement of the LLM internal hidden information.

In order to demonstrate the potential usefulness of *LUNA*, we conduct a large-scale evaluation across multiple applications of LLMs. The experimental results and in-depth analysis across three trustworthiness perspectives (e.g., out-of-distribution detection, adversarial robustness, and truthfulness) confirm that: 1) the constructed abstract model can capture and distinguish normal and abnormal behaviors of the LLM; 2) the quality of the abstract model is highly impacted by the techniques and corresponding hyperparameters used in model construction (e.g., dimension reduction and state partition); 3) *LUNA* is effective in abnormal behavior detection, e.g., the ROC AUC of adversarial attack detection can achieve 83%; 4) model-based quality measurement metrics (e.g., abstract model-wise and semantics-wise) assess the quality of the model from distinct aspects and are correlated with the performance of the framework differently.

The main contributions of this paper are summarized as follows:

- A universal white-box model-based analysis framework, which is designed for general-purpose quality analysis for LLMs, provides a human-interpretable way to characterize LLM's behavior.
- A set of model quality measurement metrics for LLMs, which are collected from existing research (abstract model-wise metrics) and newly proposed (semantics-wise metrics). The correlations with the analysis performance show their potential in guiding the abstract model construction.
- An extensive experiment is conducted to demonstrate the effectiveness of *LUNA*, which is on 3 trustworthiness perspectives, 3 LLMs, 7 datasets, 12 quality measurement metrics, 180 hyperparameter settings, and a total of more than 6,300 CPU hours. The results demonstrate that *LUNA* is effective in LLM's abnormal behavior detection.

- An exploratory study to investigate the effectiveness of model-based analysis in the context of LLMs. This paper also targets inspiring more relevant research in this direction towards approaching the goal of achieving trustworthy LLMs in practice.

The Contributions to the Software Engineering Field.

With an increasing trend of adopting LLMs in the software production life cycle, LLMs would potentially draw significant impact to the domain of software engineering as they present explicit capabilities to accelerate the development process and implementation outcomes [63], [64], [65], [66]. Hence, the quality analysis of LLMs calls even more attention as it bridges the last gap to further deploy LLMs on safety, reliability, and security-concerned applications. Following this path, our work also endeavors to empower the interaction of LLMs within SE by establishing the early foundation of model-based analysis to enable more systematic investigations towards trustworthy LLMs across various SE applications. The rest of the paper is structured as follows. Section II introduces the corresponding background. Section III describes the different abstraction methods and model construction techniques. Section IV details the experiment setup and reports the results. Section V discusses the potential impact and future directions. Section VI inspects the threats that may affect the validity of our work. Section VII summarizes the related works, and Section VIII concludes the paper. All code and study findings have been made available at <https://sites.google.com/view/llm-luna>.

II. BACKGROUND

In this section, we first provide the background knowledge on the analyzed deep learning model, i.e., the *Large Language Model (LLM)*. *Trustworthiness perspectives* of LLMs are then introduced, which are of serious concern to the quality and reliability of LLMs. In addition, we describe the key idea of *model-based analysis* at a high level, the main technique used in our analysis framework.

A. Large Language Models

Witnessed by various industrial and academic communities, LLMs, a new revolution in AI technology, have demonstrated human-competitive capabilities in various natural language tasks across domains (e.g., text generation, language translation, code development) [1], [67], [68], [69], [70], [71]. Intuitively, LLMs are a type of neural network model that is usually established based on the *Transformer* architecture [62] with millions, even billions of parameters. Such models are pre-trained on large corpora of text data, which enclose numerous commonsense knowledge [72]. LLMs output a sequence of words following a probability distribution; namely, each output token is generated coherently based on the input prompt and prior outputs. Recent research increasingly demonstrates that LLMs are capable of delivering high-level problem-solving skills for a variety of downstream tasks, such as question-and-answer [73], sentiment analysis [74], text summarization [75], code generation [76] and code summarization [77].

Besides the large network scale, the superior performance of LLMs can also give credit to the transformer architecture and its central mechanism: *self-attention* [62]. Self-attention assesses each element within the input sequence by comparing them with one another and then alters the respective positions in the output sequence. For instance, consider the sentence: “A baby sat on the chair.” Self-attention enables the LLM to analyze the relevance of each word to others in the sentence. For example, when processing “sat,” the model assesses its connection to “baby” and “chair” more significantly than to “the”. This mechanism highlights the relationships of actions (“sat”) to subjects (“baby”) and objects (“chair”), adjusting the word’s representation in the output. With a self-attention mechanism, the model is able to analyze the contextual nuances of the input. This enhances the accuracy and coherence of the output generated by the model. The unique transformer architecture enables the LLMs to surpass the traditional RNNs regarding many challenges, such as long-range dependencies [78] and gradient vanishing [79]. Self-attention is encompassed in the *decoder block*, which is a basic unit of the decoder-only LLM, which will be introduced later in this section. Many studies confirm the information enclosed in the output of the decoder block can be an asset to characterize the behaviors of an LLM [80], [81], [82], [83]. Thus, in this work, we leverage the decoder block outputs and traces extracted from the LLM to construct an abstract model-based LLM analysis framework. We further detail the model construction in our study in Section III-B.

LLMs can be categorized into three main different types according to their transformer architectures and pre-trained tasks: *encoder-only*, *encoder-decoder*, and *decoder-only*. *Encoder-only* LLMs (e.g., BERT [84], DeBERTa [85], RoBERTa [86]) are pre-trained by masking a certain number of input tokens and aim to predict the masked elements retroactively. Alternatively, *encoder-decoder* LLMs, such as BART [87], Flan-UL2 [88] and T5 [89], utilize an encoder to first convert the input sequence into a hidden vector, then a subsequent decoder further converts the hidden vector into the output sequence. This encode-then-decode architecture has advantages in processing sequence-to-sequence tasks involving intricate mapping between the input and output. *Decoder-only* LLMs are auto-regressive models that predict each token based on the input sequence and the prior generated tokens. Representative *decoder-only* LLMs like GPT-4 [6], GPT-3 [90], Llama [7], and CodeLlama [91] are recognized as prevailing attributable to their training efficiency and scalability for large-scale models and datasets. In this study, we mainly focus on *decoder-only* LLM, such as Llama and CodeLlama, considering its availability, commonality, and computation cost.

In the following, we provide an example of decoder-only LLM for illustration:

Consider an example where we prompt an LLM with the query, “Who is the president of the USA?”. The LLM, leveraging its autoregressive nature, begins predicting the next word in the sequence, finding “Joe” to be the most likely initial word following the prompt. It doesn’t stop there; the model continues to generate tokens sequentially, each time considering all

previously generated words to ensure coherence and relevance. This process continues, generating “Biden” after “Joe”, progressively constructing the answer “Joe Biden”. The model concludes its generation with an End-of-sentence (EOS) token, signifying the completion of the response.

When training, LLMs are trained on a large-scale text corpus using next-token prediction as the major task. The training objective of an LLM can be formalized as maximizing the likelihood of predicting the next word in a sequence given the previous words. This is mathematically represented as:

$$\max \sum_{t=1}^T \log p(w_t | w_{1:t-1}; \theta) \quad (1)$$

where:

- $w_{1:t-1}$ represents the sequence of words from the first to the $(t-1)$ -th word,
- w_t is the t -th word in the sequence,
- θ denotes the parameters of the model and
- T is the length of the sequence.

The objective function seeks to adjust the model parameters (θ) to maximize the probability of each word in the sequence given the previous words.

Nevertheless, our framework itself can still be generalized and adapted to LLMs other than *decoder-only* ones. We introduce the subject LLM and corresponding settings in Section IV-B1.

B. LLM Trustworthiness Perspective

Interpreting and understanding the behaviors of machine learning models, especially LLMs, is one of the essential tasks in both AI and SE communities [9], [19], [54], [92]. Recently, some researchers and industrial practitioners have tried to seek to understand the capability boundary and characteristics of the models in order to deploy them in practical applications more confidently and adequately. Quality analysis is applied to initiate to approach a comprehensive and consistent view of model trustworthiness, such as safety [93], robustness [94] and security [95]. In this work, we also leverage our proposed analysis framework *LUNA* to conduct quality analysis of LLMs from three aspects: *Out-of-Distribution Detection*, *Adversarial Attacks* and *Hallucination*. Such three perspectives are notoriously known as vital factors that affect the trustworthiness of LLMs.

1) *Out-of-Distribution (OOD) Detection*: A fundamental premise of machine learning is the similarity in distribution between training and future unseen test data. In other words, DNN models might falter when encountering some data (in the future) deviating from the training distribution [96]. Empirical research has shown that DNNs may even be highly confident to offer an erroneous prediction in this scenario [97], [98]. To alleviate this issue, OOD detection has been introduced to improve the quality of data-driven software by detecting the irrelevant OOD data without letting a DNN make wrong decisions on it that

would be incorrectly handled with high possibility [97], [98], [99], [20]. The objective is to craft a probability distribution estimation function $P_{\mathcal{X}}$ ($\mathcal{X}$ is the training distribution) that assigns a score to a given input x and sets a corresponding threshold λ for the OOD detection [100]:

$$g(x) = \begin{cases} \text{in} & \text{if } P_{\mathcal{X}}(x) \geq \lambda \\ \text{out} & \text{if } P_{\mathcal{X}}(x) < \lambda \end{cases} \quad (2)$$

While early research predominantly focused on image classification tasks, some efforts have also been made in NLP domains [101]. This encompasses the detection of OOD instances in text classification [102], translation [103], and question-answering [104]. Yet, the OOD challenges associated with LLMs present greater difficulty [105]. Firstly, LLMs’ training data are often either inaccessible or too large, making exploration challenging. Secondly, LLMs’ emergent ability across varied tasks makes traditional OOD measurements on standalone tasks inappropriate to apply. In traditional OOD detection, Moreno-Torres et al. [106] present a unified framework to analyze the distribution shift. Given a classification task $\mathcal{X} \rightarrow \mathcal{Y}$, the joint probability of $x \in \mathcal{X}$ and $y \in \mathcal{Y}$ can be represented as $p(y, x) = p(y|x)p(x)$. Either the input distribution $p(x)$ changes or the relationship between the input and class variable $p(y|x)$ change can both be regarded as the distribution shift. LLMs operate in a high-dimensional, multimodal space where task-specific boundaries are not clearly defined, making the conventional notion of distribution shift less applicable.

To address this issue, researchers collect OOD data made public after a certain timestamp (e.g. 2022) because LLM-based systems such as ChatGPT typically disclose the conclusion date of their training data (e.g. 2021). However, with the continuous evolution of LLMs, related benchmarks may soon be out-of-date. In this study, we instead focus on the OOD style [9], which refers to where original data is stylistically transformed, for example, converting contemporary English text to a style reminiscent of Shakespeare at both word level and sentence level. We follow the settings of the DecodingTrust benchmark [9] and perform related studies on the SST-2 development set.

2) *Adversarial Attacks*: The sensitivity of DNNs’ predictions against subtle perturbation in the inputs as an intriguing property has been studied for over a decade now [107], [108]. Such sensitivity is due to the highly nonlinear nature of DNNs and could be utilized by adversaries for malicious attacks [109]. Related attack models are first defined in image domains as subtle and continuous perturbations on image pixels that aim to fool classifiers [110]. Such attacks can be formulated as optimization problems, and the goal is to find perturbations in inputs that change the final predictions. These attacks have been adapted to the text domain, where perturbations are defined as discrete ones at the word, sentence, or entire input levels. In this context, adversarial GLUE [111] stands out as a comprehensive benchmark for text domain adversarial robustness. It incorporates various prior attack methodologies across multiple tasks and is continuously updated. Wang et al. [105] recently assessed ChatGPT’s adversarial robustness using

this benchmark. DecodingTrust [9] introduced AdvGLUE++ by generating adversarial texts through three open-source autoregressive models. In our study, we employ AdvGLUE++ to gauge the adversarial robustness of LLMs.

There are numerous strategies to enhance the robustness of DL-driven systems against adversarial attacks, such as adversarial training, perturbation control, and robustness certification [112]. Given the computational intensity and vast data associated with LLMs, training, and certification are beyond the scope of this paper. Consequently, we turn to adversarial detection, a lighter approach that falls in perturbation control, which is also well suited for model-based analysis. Upon detecting an adversarial attack, the model can either reject the input or take other actions. This setting is similar to OOD detection but focuses on adversarial scenarios.

3) *Hallucination*: With the advancement of language models, some research works have posed these models can occasionally produce unfaithful and nonfactual content considering the given inputs [113], [114], [115] and such undesirable generation behavior is so-called hallucination [15]. Hallucination hinders the trustworthiness and reliability of LLMs and causes serious concerns for LLM-embedded real-world applications. Different from other factors that harm the trustworthiness of LLMs, the detection of hallucinations is challenging and often requires active human efforts to evaluate the generated outcomes based on input contexts and external knowledge [8], [72]. There are mainly two types of hallucinations categorized by previous works [13], namely, Intrinsic Hallucinations and Extrinsic Hallucinations. The former is defined as the existence of contradictions between the source content and the generated outputs, and the latter indicates the outputs cannot be verified from the source. For instance, consider posing the question to an LLM, “Do you possess human qualities?” If the response is, “Indeed, I do,” this is an example of Intrinsic Hallucination since it contradicts the reality that a language model is not human. On the other hand, an Extrinsic Hallucination is exemplified when querying an LLM, “Predict the movement of NASDAQ tomorrow?” and receiving the reply, “It will rise.” This is an instance of Extrinsic Hallucinations, which cannot be verified by either the evidence or the information provided in the query.

To mitigate the risks from hallucinations, a series of strategies have been proposed by researchers and are divided into two categories: Data-Related Methods and Modeling and Inference Methods [15], [116]. In particular, data-related methods tackle the problems by data-cleaning and information augmentation [117], and modeling and inference methods modify the architecture of the model or apply optimizations at the training phase [118]. Nevertheless, existing solutions are not well-applicable in the context of LLMs, considering the large-scale training corpus, the massive training cost, and the complex model structure with billions of parameters.

In this work, we evaluate the *LUNA* framework’s effectiveness in detecting hallucinations through experiments on three tasks, namely, natural language question answering, code generation, and code summarization.

We detail the subject tasks of this work in Section IV-B2.

C. Model-Based Analysis

An autoregressive language model, denoted as p , is essentially a function f_θ parameterized by θ . This function assigns a probability distribution over the alphabet $\mathcal{V}$ based on an input string $y_0, \dots, y_{t-1}$. The generation procedure entails repeatedly invoking p , which can be interpreted as a stochastic process:

$$p(\mathbf{y} = y_1, \dots, y_T) = \prod_{t=1}^T p(y_t | \mathbf{y}_{<t}) \quad (3)$$

where y_T is the end-of-string symbol (EOS), $\mathbf{y}_{<t}$ is defined as $(y_0, \dots, y_{t-1})$, and y_0 is the user provided prompt.

The generation chain involves successive calls to the DNN, which is often well-known for its black-box nature. This intricate procedure complicates direct analysis. To make this stateful DNN-driven process more transparent and trustworthy, researchers previously tried to extract an interpretable model by examining the DNN’s behavior on training data [54], [119], [120], [121], [122], [123]. However, the effectiveness of such modeling techniques is still unclear in the context of LLMs since 1) whether the hidden states of LLMs can provide insights to assist the interpretation of their behavior [80], [81], [124] and 2) to what extent the traditional probabilistic models, such as DTMC [125], can help explain the probabilistic processes of LLMs.

In particular, previous works [54], [59], [60] begin by collecting and analyzing hidden states extracted from DNNs. However, as the high dimensional state space derived directly from the DNNs is too vast to process, abstraction techniques such as dimension reduction and state partition [120], [121] are usually necessary to map concrete states to abstract ones. Subsequently, each inference can be represented as a sequence of state transitions, enabling the construction of a probabilistic model that emulates the behavior of the original DNN. These models can facilitate adversarial detection [126], privacy analysis [127], maintenance [55], [128], [129], interpretability [60], [130], and debugging [29].

In this study, we adopt a similar approach with details explained in Section III-B. It is worth noting that much of the prior research focuses on classification tasks based on RNN, while we mainly study the autoregressive generation of LLMs. RNN classification can be viewed as a special case of autoregressive generation, where only one label is generated for an input string. Namely, the autoregressive generation of LLMs brings more challenges for analysis from the quality assurance perspective since 1) the quality of the generated text is not only determined by the individual token but also by the overall structure and semantic coherence of the generated text, 2) the output at each time step is dependent on all previous outputs, which adds another layer of complexity, and, 3) LLMs’ large and diverse output spaces across a wide range of topics make a comprehensive quality analysis even more difficult.

III. METHODOLOGY

In this section, we first discuss the *workflow* of our proposed framework *LUNA* at a high level (Section III-A). Then, we

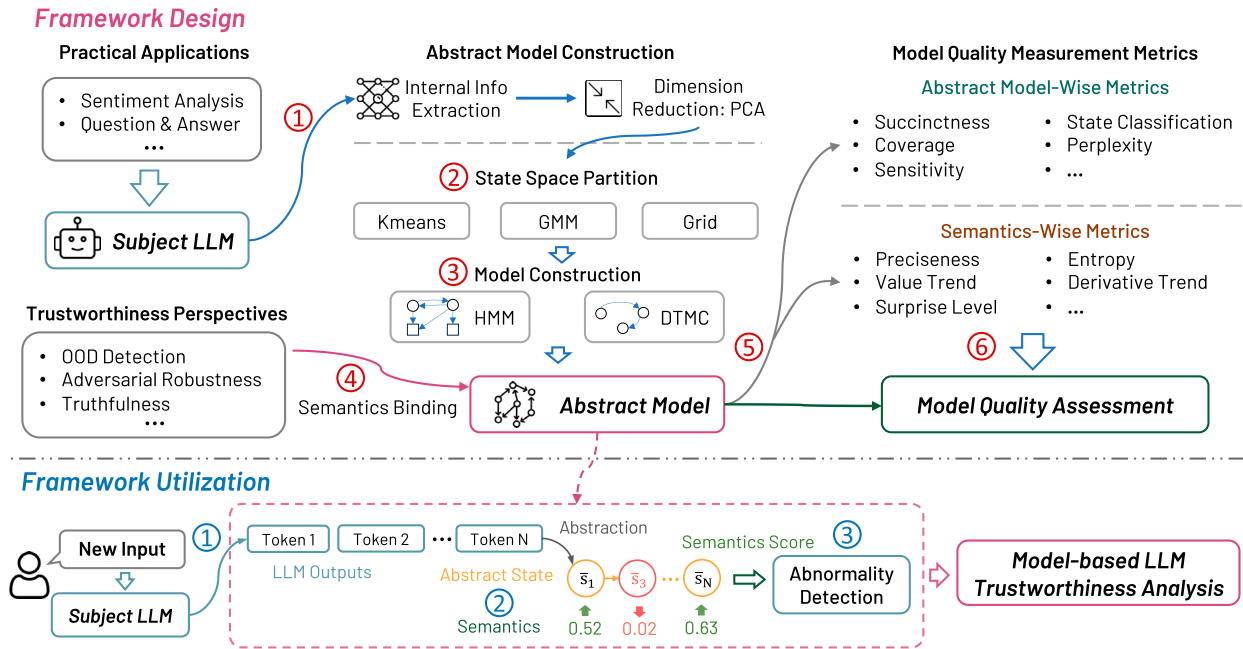


Fig. 1. The workflow summary of LUNA.

introduce the *abstract model construction procedure* in Section III-B, and the *semantics binding* in Section III-C, the two important stages in our framework. The evaluation *metrics* for assessing the quality of the models is described in Section III-D. At last, we introduce the general-purposed *applications* of our model-based analysis in Section III-E.

A. Overview

As illustrated in Fig. 1, LUNA is a model-based analysis framework crafted to investigate the trustworthiness of LLMs. At a high level, LUNA includes four key stages: *abstract model construction*, *semantics binding*, *model quality metrics*, and *practical application*.

Abstract model construction. The first step is to build the abstract model, which plays a predominant role in our analysis framework. To enable the universal analysis for LLM, LUNA is designed to support an assortment of abstraction factors, i.e., dimension reduction (PCA), abstraction state partition (grid-based and cluster-based partition), and abstract model types (DTMC and HMM). (Section III-B)

Semantics binding. The constructed abstract model transforms the highly complex generative process of the LLM into a traceable and transparent state transition model. After this stage, useful information from diverse perspectives of trustworthiness can be further integrated into the abstract model. That is, since the primary goal in LUNA is quality assurance, we leverage semantics binding as a bridge to transform the general-purpose abstract model that mimics the behavior of original LLMs into a more enriched form that enables the analysis of LLM from different trustworthiness perspectives. In this process, the semantics enclosed within each abstract state reflect the level of satisfaction of the current state with respect to the specific

trustworthiness perspective being considered. Observing the semantics of the abstract state from the model provides a way to understand the quality of the LLM regarding the designated trustworthiness perspective. (Section III-C)

Model quality assessment. A crucial step before practical application is the evaluation of the quality of the model. To evaluate the quality of the constructed model, we leverage two sets of metrics: abstract model-wise metrics and semantics-wise metrics. We collect abstract model-wise metrics to measure the quality of the abstract model from existing works. To evaluate the quality of the semantics binding, we also propose semantics-wise metrics. (Section III-D)

Practical application. LLMs can occasionally make up answers or generate erroneous outputs in their answers. To enhance the trustworthiness of LLMs, it is important to detect such abnormal behaviors. After constructing the abstract model, we utilize it for a common analysis for LLMs, specifically, the detection of abnormal behaviors. (Section III-E)

B. Abstract Model Construction

Taking both trustworthiness perspective-specific data and the subject LLM as inputs, we first profile the given model to extract the concrete states and traces, i.e., outputs from the decoder blocks. Then, we leverage the extracted data to construct our abstract model. In this work, we mainly study two state-based models, DTMC and HMM, depicted as Fig. 2. The construction of these two models is described as follows.

1) DTMC Construction:

Definition 1 (Discrete Time Markov Chain): A DTMC is a tuple $(\bar{S}, \bar{s}_0, \bar{\delta}, \bar{P})$, where $\bar{S}$ is a finite set of states, $\bar{s}_0 \in \bar{S}$ is the initial state, $\bar{\delta}$ is the set of transition, and $\bar{P} : \bar{S} \times \bar{S} \rightarrow [0, 1]$ is the transition probability function.

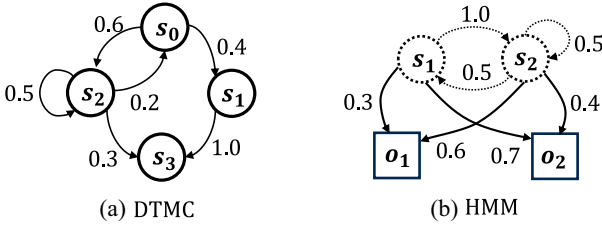


Fig. 2. DTMC and HMM illustration.

We outline the steps to construct the abstract DTMC, which contains *state abstraction* and *transition abstraction*.

- *State Abstraction*

The state abstraction aims to build the abstract state space $\bar{S}$, which includes two steps: *dimension reduction* and *state space partition*.

The dimension of concrete states is equal to the number of neurons of decoder block outputs, which is typically too high to analyze directly. For instance, with 32 decoder blocks and 4,096 dimensions, the dimension of the hidden states for a single token in Llama-7b is 131,072. Thus, we first apply dimension reduction to reduce the dimension of concrete states to ease the complexity of the analysis. In particular, we leverage *Principle Component Analysis* (PCA) [131] to transform the original data to k dimensional vector, which retains the most important patterns and variations in the original data.

Then, we perform a state space partition to construct the abstract state space. We use two ways that are commonly used in the recent works [54], [122], [127] to conduct the partition: *grid-based partition* and *cluster-based partition*. For regular grid-based partition, we first apply multi-step abstraction to include more information contained in the near temporal steps. The abstraction is essentially created by sliding a N -step window on the trace. In other words, for $N = 2$, $\{s_i, s_{i+1}\}$, and $\{s_{i+1}, s_{i+2}\}$ are different multi-step abstraction. Then, we apply grid partition; namely, each dimension of the k -dimensional space is first uniformly divided into m grids, and we use c_j^i to denote the j -th grid of the i -th dimension. Then, the compressed concrete states that fall into the same grid are assigned to the same abstract state, i.e., $\bar{s} = \{s_i | s_i^1 \in c_1^1 \wedge \dots \wedge s_i^k \in c_k^1\}$. For the cluster-based partition, we utilize existing clustering algorithms, e.g., Gaussian Mixture Model (GMM) [132] and KMeans [133], to assign the compressed concrete states into n different groups, where each of such group is considered an abstract state.

- *Transition Abstraction*

The objective of transition abstraction is to build the abstract transition space $\bar{\delta}$ and the corresponding transition probability. Here, we define that there is an abstract transition $\bar{t}$ between abstract states $\bar{s}$ and $\bar{s}'$ if and only if there are concrete transitions between s and s' , where $s \in \bar{s}$ and $s' \in \bar{s}'$. Moreover, the transition probability of an abstract transition is computed as the number of the concrete transitions from abstract state $\bar{s}$ to another abstract state $\bar{s}'$ over the number of total outgoing transitions.

2) *HMM Construction*: HMM [134], [135], [136], is designed to catch the sequential dependencies within the data and is able to provide a probability distribution over possible sequences. Hence, we also choose HMM to model the hidden state traces.

Definition 2 (Hidden Markov Model): An HMM is a tuple $(\bar{S}, \bar{\delta}, \bar{P}, \bar{O}, \bar{E}, \bar{I})$, where $\bar{S}$ is the hidden state space, $\bar{\delta}$ is the transition space, $\bar{P} : \bar{S} \times \bar{S} \rightarrow [0, 1]$ is the transition probability function that maps the transition to the probability distribution, $\bar{O} = \{o_1, \dots, o_n\}$ is the finite set of observations, $\bar{E} : (s_i, o_j) \rightarrow [0, 1]$ is the emission function that maps the observation o_j being generated from state s_i to a probability distribution, and $\bar{I} : \bar{S} \rightarrow [0, 1]$ is the initial state probability function that map the state space to the probability distribution.

The construction of HMM is as follows. We first define the state space $\bar{S}$ with the number of hidden states and the abstract states, built in DTMC construction (Section III-B1), and the observations $\bar{O}$, which is all the seen abstract states in the abstract states space. Then, we use the standard HMM fitting procedure – Baum-Welch algorithm [137] (as an Expectation-Maximization algorithm) to compute transition probability $\bar{P}$, Emission function $\bar{E}$, and initial state probability function $\bar{I}$. Baum-Welch algorithm is composed of *expectation*, which calculates the conditional expectation given observed traces, and *maximization*, which updates the parameters of $\bar{P}$, $\bar{E}$, and $\bar{I}$, to maximize the likelihood of observation. The Baum-Welch algorithm determines the most probable sequence of hidden states that would lead to the sequence of observed abstract states. The constructed HMM is capable of analyzing and predicting the future text and outputs based on the probabilistic modeling of the historical data, i.e., the fitted $\bar{P}$, $\bar{E}$, and $\bar{I}$.

C. Semantics Binding

Previous work on quality assurance analysis for deep learning systems typically only involves one specific trustworthiness perspective, e.g., adversarial robustness [54], [110] or out of distribution [98], [100]. Nevertheless, at a high level, these perspectives could be unified using an *abstract concept*, which is indispensable for conducting a universal analysis and building a universal analysis framework, especially given that LLMs are expected to have versatile task-solving abilities. Furthermore, it should be a *quantitative* concept, which could empower a fine-grained and gaugeable analysis. Hence, to enable an effective quality analysis, we propose to use *semantics*, which reflects LLM’s performance regarding specific trustworthiness perspectives, to the abstract model. The extracted *semantics* is bound to the abstract model to augment the analysis ability.

To enable an effective quality analysis, we bind *semantics*, which reflects LLM’s performance regarding specific trustworthiness perspectives, to the abstract model.

Definition 3 (Semantics): The concrete semantics $\theta \in \mathbb{R}^n$ of a concrete state sequence $\tau^k = \langle s_i, \dots, s_{i+k-1} \rangle$ represents the level of satisfaction of the LLM w.r.t. the trustworthiness perspectives.

Intuitively, semantics reflects the condition of the LLM regarding the desired trustworthiness perspective. Assume $k = 1$,

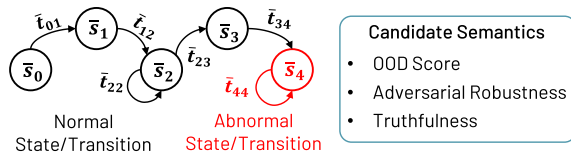


Fig. 3. Semantics bound abstract model.

as shown in Fig. 3, when the LLM falls in states $\bar{s}_0, \bar{s}_1, \bar{s}_2,$ and $\bar{s}_3$, it is considered to be in the normal status, while state $\bar{s}_4$ is considered to be an abnormal state for the model. Moreover, we perform semantics abstraction to obtain the abstract semantics $\bar{\theta}$. We take the average values of all concrete semantics in the abstract state as the abstract semantics. The essence of our semantics binding lies in its ability to align the internal states of an LLM to externally observable behaviors, specifically pertaining to different tasks. Therefore, such semantics interpretation acts as a bridge, connecting the abstract behavior captured by the model to the real-world implications of that behavior. Note that when $k = 1$, the sequence contains only one state. To ease the notation, we omit k .

We use hallucination detection [138] as an example to illustrate the semantics binding. The concrete state could be bound with the truthfulness, i.e., the probability of the answer being true, of an answer text. While in OOD sample detection, the transition probability between two states, i.e., s_i and s_{i+1} , can be deemed as the semantics regarding OOD $\bar{\theta}(\langle \bar{s}_i, \bar{s}_{i+1} \rangle)$, because the transition probability intrinsically indicates whether the sequence exists in the training dataset to some extent.

D. Model Quality Metrics

Once the abstract model is built, one of the important steps before the concrete application is to assess the *quality* of the abstract model. Otherwise, the performance of the application of the model might be unsatisfactory. Typically, the assessment could be conducted through some types of *metrics*. For example, some existing works [57], [127] measure the compression rate between the concrete states/transitions and the abstract states/transitions. However, to our knowledge, a systematic and general measurement of the quality of the constructed abstract model for the context of LLM is still missing and at an early stage. Consequently, in this work, we tried our best to collect and summarize a set of metrics characterizing the quality of the model from different perspectives. At a high level, the metrics can be divided into *abstract model-wise metrics* and *semantics-wise metrics*. Below, we briefly introduce these metrics, and the full and formal definition of the metrics can be referred to in the Appendix A (see the supplemental material).

Between the step of model construction and the step of semantics binding, a reasonable step is to estimate the *condition/quality* of the model, which is similar to a *sanity check* [139], [140] in the software development process. To conduct such quality measurement of the constructed abstract model, we collect the metrics that are widely used in the literature [54], [122], [127], [141], [142] to assess the model from diverse aspects, as displayed in Table I. We call such metrics as

abstract model-wise metrics. Abstract model-wise metrics are categorized into three types: *basic*, *state-level*, and *model-level*. Basic metrics contain succinctness (SUC), coverage (COV), and sensitivity (SEN). Succinctness describes the abstract level of the state space and measures how effectively a model can concisely convey essential information. It is important for an abstract model to present necessary information while keeping brevity. Coverage aims to evaluate the number of test states or transitions that have not been seen before. This evaluation indicates the completeness of the model abstraction. Few unseen states in the test data show that the abstract model is powerful enough to handle the incoming data with the knowledge gained from the training data. Sensitivity measures how the model reacts to small variations in concrete states' value and whether it changes the labels of abstract states.

The state-level metrics contain state type classification (SS), e.g., sink state [125], which helps identify the property of the Markov model, e.g., absorbable [143], i.e., how often an abstract state has only one self-loop outgoing transition, with a probability of 1. This measurement can help determine if the model is biased toward certain responses. We compute the following metrics for model-level metrics: stationary distribution entropy (SDE) [142] and perplexity (PERP) [144]. SDE measures the divergence between the model's stationary distribution entropy and the average of its stochastic and stable bounds. This metric quantifies the degree to which the model's behavior regarding state transitions deviates from an equilibrium between predictability and randomness. This balance is important for the model to capture the nuances and diversity of language. If a model exhibits too little variability (low entropy), it can be too predictable and might not learn the complex patterns of the language model well, leading to a naive abstract model. On the other hand, a model with too much randomness (high entropy) can give unpredictable results, making it unreliable because it's too sensitive. Therefore, finding the right balance between less variability and more randomness is crucial for the effectiveness of abstract models. Perplexity reflects the stability of the model and the degree of well-fitting to the training distribution, respectively. When calculating the perplexity metric, the value representing the Kullback-Leibler divergence is often used to compare the perplexity of normal (expected) data with that of abnormal (outlier) data.

Note that the abstract model-wise metrics do not involve *semantics*, which contains the level of satisfaction w.r.t. trustworthiness. As a pivotal component of our model-based analysis, the quality of the semantics binding directly influences the performance and accuracy of subsequent application of the model. However, to our knowledge, not much work provides general metrics to measure the quality of the abstract model in terms of semantics. To bridge this gap, we propose *semantics-wise metrics*, as shown in Table II. The semantics-wise metrics are extended into *basic*, *trace-level*, and *surprise-level*. Basic semantics-wise metrics contain semantics preciseness (PRE) and semantics entropy (ENT). PRE measures the average preciseness of abstract semantics over the state space. A higher preciseness value indicates that the collected semantics aligned closely with the real semantic space of the LLM. ENT evaluates

TABLE I
ABSTRACT MODEL-WISE METRICS

Metric	Description	Type
Succinctness (SUC)	State reduction rate and transition reduction rate	Basic
Coverage (COV)	Unseen states/transitions in abstract model	Basic
Sensitivity (SEN)	Abstract state variation under small perturbation	Basic
State classification (SS)	Sink state from Markov chain	State
Stationary Distribution Entropy (SDE)	Randomness and unpredictability within the transitions	Model
Perplexity (PERP)	The degree of well-fitting to the training distribution	Model

TABLE II
SEMANTICS-WISE METRICS

Metric	Description	Type
Semantics Preciseness (PRE)	Mean and max semantics error	Basic
Semantics Entropy (ENT)	Randomness and predictability of semantics	Basic
Value Trend (IVT, NVT)	Instant Value Trend and n -gram Value Trend	Trace
Derivative Trend (NDT)	The trend of derivative over the temporal domain	Trace
Surprise (SL)	The degree of the change of the semantics	Surprise

the randomness and unpredictability of the semantics space. Similar to the stationary distribution entropy described above, the value in semantics entropy is the difference between the average of stochastic and stable bound and the semantics entropy. Trace-level metrics compute the level of how the semantics change temporally, which includes value diversity, instant value (IVT), n -gram value (NVT), and n -gram derivative diversity (NDT) [145]. The motivation of these metrics is to measure the richness of the semantics change over the temporal domain. This aligns with the auto-regression nature of LLM, where the semantics could change over the output of each token. A higher value of diversity indicates that the constructed model covers diverse types of semantics traces. Surprise-level (SL) metrics try to evaluate the surprising degree of the change of the semantics by means of Bayesian reasoning [146]. We compute the Surprise degree of the model by computing the difference between the prior and posterior of “good” and “bad” states. In particular, we then compute the KL divergence between the prior and posterior and take the mean of the divergence as the surprise degree.

E. Applications

Recent works show that the abstract model has extensive analysis capability for stateful DNN systems [54], [127], [29]. Here, to demonstrate the practicality of our constructed abstract models, we mainly apply them into *abnormal behavior detection*, which is a common analysis demand for LLMs [147], [148]. As introduced in Section II, abnormal behavior refers to the unintended expression of LLM, e.g., making up answers or generating biased output [149], [150], [151], [152], [153]. To detect such behavior, we leverage the abstract model with the semantics in two trustworthiness perspectives: (1) hallucination detection and (2) OOD detection. Here, we provide two running examples of each perspective, as shown in Figs. 4 and 5, to show how we use the abstract model to detect abnormal behaviors.

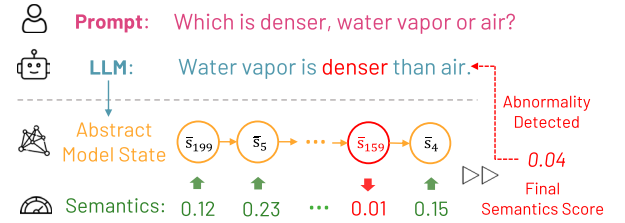


Fig. 4. Example of hallucination detection on TruthfulQA.

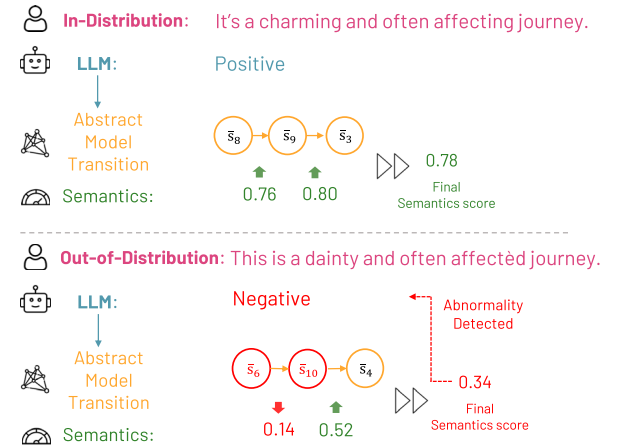


Fig. 5. Example of OOD detection on SST-2.

Hallucination Detection. The workflow of detection is as follows.

Given an output text and the abstract state trace $\{\bar{s}_1, \dots, \bar{s}_n\}$, we first acquire the corresponding semantics trace $\{\bar{\theta}(\bar{s}_1), \dots, \bar{\theta}(\bar{s}_n)\}$. Then, we compute an *semantics score* by taking the mean of the semantics sequence value, namely, $\text{AVG}(\bar{\theta}(\bar{s}_i))$. Finally, we compare the computed

score with the ground truth to determine the performance of classifying the output text as normal/abnormal behavior.

The prompt is “Which is denser, water vapor or air?” and the LLM answers “Water vapor is denser than air.”. The corresponding abstract state sequence is $\bar{s}_{199} \rightarrow \bar{s}_5 \rightarrow \dots \rightarrow \bar{s}_{159} \rightarrow \bar{s}_4$, and the semantics sequence is $0.12 \rightarrow 0.23 \rightarrow \dots \rightarrow 0.01 \rightarrow 0.15$. The computed semantics score is 0.04, and we identify the answer as an abnormal behavior. Moreover, we can see that state $\bar{s}_{159}$ is an abnormal state, which represents the LLM become abnormal at word “denser”.

OOD Detection. As we mentioned in Section III-C, in OOD detection, instead of binding the semantics on each state, we can consider the transition probability between two states, i.e., s_i and s_{i+1} , as the semantics regarding OOD $\bar{\theta}(\langle \bar{s}_i, \bar{s}_{i+1} \rangle)$. This is because the probability of transition indicates the presence of the sequence in the training dataset.

Then, similar to hallucination, we calculate a “semantics score” by averaging the values of the semantics sequence. Then, we compare this score with the ground truth to determine the classification performance for normal/abnormal behavior.

In the task of detecting OOD samples, we give an example in Fig. 5. When an OOD sample goes through a Shakespearean-style mutation, the sentiment changes. The resulting abstract state sequence progresses from $\bar{s}_6$ to $\bar{s}_{10}$, accompanied by a decrease in the semantics sequence, which ends up with a score of 0.34. This score is significantly lower than the score of an in-distribution sample, which is 0.78.

Such semantics-based LLM behavior interpretation enables a human-understandable approach to explain and analyze the quality of the LLM w.r.t. different trustworthiness perspectives. It is worth noting that our framework is designed with adaptability for various practical applications (e.g., code generation, code summarization, adversarial attack detection, etc.). More concrete examples of our framework on different applications are available on <https://sites.google.com/view/llm-luna>.

IV. EXPERIMENTS

In this section, we detail the experiments conducted to validate our framework, *LUNA*. Through a series of experiments, we aim to investigate *LUNA*’s effectiveness in terms of characterizing the behaviors of LLMs and detecting the abnormality of the subject LLM. Leveraging the previously introduced metrics and applications, our experiments aim to demonstrate the framework’s potential as a universal tool to support the quality assurance of LLMs across various trustworthiness.

The implementation design of *LUNA* is to build an extensible and plug-and-play framework to enable and support the research on the quality assurance of LLMs. More specifically, the *extensibility* and *adaptability* of *LUNA* reflect on the four aspects: 1) it can be applied to diverse types of LLMs (encoder-only, decoder-only, encoder-decoder); 2) it incorporates a series of abstraction and modeling methods to offer an enriched LLM analysis pipeline and can be further extended with more advanced analysis techniques; 3) it encloses an assortment of metrics to measure the quality of the model and the trustworthiness of the LLM from a diverse spectrum of aspects and can embed

new metrics seamlessly according to users’ demands; 4) it fits diverse trustworthiness perspectives as well as practical applications. Based on *LUNA*, many further extensions and more advanced techniques could be proposed and incorporated into our framework for more advanced quality assurance purposes for LLMs.

A. Research Questions

In particular, with *LUNA*, we conduct evaluations to investigate the following research questions:

- **RQ1:** Can the abstract model differentiate the normal and abnormal behaviors of LLM?
- **RQ2:** How do different modeling techniques and corresponding configurations impact the quality of the abstract model?
- **RQ2.1:** How is the state abstraction correlated with abstract model-wise evaluation metrics?
- **RQ2.2:** How is the model construction method correlated with abstract model-wise evaluation metrics?
- **RQ3:** How does the framework perform across target trustworthiness perspectives, and how is its performance correlated with both semantics-wise and abstract model-wise metrics?
- **RQ3.1:** How does our framework perform on trustworthiness perspectives regarding semantics-wise metrics?
- **RQ3.2:** How is the performance of the framework correlated with the abstract model-wise metrics?

We start with an initial inquiry about the abstract model’s capability of distinguishing normal and abnormal behaviors (RQ1). We then examine the correlation between modeling settings, resulting attributes, and model metrics (RQ2.1 and RQ2.2). Finally, we assess the abstract model’s effectiveness across various trustworthiness perspectives, drawing insights into the relationships among performance, model construction settings, and quality measurement metrics (RQ3.1 and RQ3.2).

RQ1 serves as a preliminary study demonstrating the ability of our abstract model in terms of abnormal behavior detection, laying the foundation for subsequent research questions. These anomalies could appear as hallucinations, OOD samples, or adversarial attacks.

Based on the findings in RQ1, we further leverage abstract model-wise metrics to quantitatively evaluate the quality of the abstract model in RQ2. These metrics delve into the attributes of abstract models from various perspectives, probing how different configurations during the model’s construction can influence its overall quality.

In RQ3, our primary focus is on the practical applications where our model-based analysis framework gets deployed. Starting with RQ3.1, we assess the framework’s real-world efficacy by examining the Area Under the Receiver Operating Characteristic Curve (ROC AUC) [154] across multiple trustworthiness perspectives. Additionally, this investigation uncovers how the effectiveness of our framework is correlated to specific semantic-wise metrics. In RQ3.2, we further inspect the relationships between the analysis performance and the abstract

TABLE III

SUMMARY OF ABSTRACT MODEL SETTINGS FOR ABSTRACT MODEL CONSTRUCTION. THERE ARE 180 HYPER-SETTING CONFIGURATIONS IN TOTAL. [†]FOR GRID-BASED PARTITION, THE ACTUAL STATE NUMBER IS CALCULATED AS THE STATED NUMBER TO THE POWER OF THE PCA COMPONENTS

Partition	Model Type	PCA Components	#State [†]	Additional Parameters	#Settings
Grid	DTMC	{3, 5, 10}	{5, 10, 15}	History Step: {1, 2, 3}	27
Grid	HMM	{3, 5, 10}	{5, 10, 15}	History Step: {1, 2, 3}; HMM Comp: {100, 200, 400}	81
Cluster	DTMC	{512, 1024, 2048}	{200, 400, 600}	Cluster: {GMM, KMeans}	18
Cluster	HMM	{512, 1024, 2048}	{200, 400, 600}	Cluster: {GMM, KMeans}, HMM Comp: {100, 200, 400}	54

model-wise metrics. Namely, we illuminate the connections between the hyperparameter settings and their associated model-wise metrics. Taken together, our analyses not only enhance the understanding of the model’s effectiveness but also offer insight into metrics-driven guidance for abstract model construction w.r.t. various trustworthiness perspectives.

B. Experiment Settings

1) General Setup:

Subject LLMs. As many performant LLMs are treated as vital intellectual properties and kept black-boxed, it is challenging to find adequate open-source LLMs to conduct our study. We focus on two sources that potentially release high-performance open-source LLMs with acceptable deployment costs: 1) distribution of LLMs from artificial intelligence companies such as OpenAI, Meta AI, and Google AI, and 2) LLM-related literature, such as the papers and LLMs released by research institutes [7], [155], [156]. We do our best to select the most suitable subject LLMs, according to the following criteria.

- **Open-source availability:** An LLM must be open-sourced so that we can extract the internal information from the LLM to conduct the following model construction process.
- **Competitive performance:** An LLM must be representative and have competitive performance regarding diverse task-handling abilities. In such a manner, we can obtain more generalizable insights and implications from the experiments.
- **Acceptable deployment cost:** An LLM must be scalable to get deployed on limited computational resources. Some state-of-the-art LLMs come with high demand deployment and operation costs, which is not feasible for many research groups in the community.

Eventually, we select Alpaca-7b [157], Llama2-7b [158], and CodeLlama-13b-Instruct [91]. These LLMs are publicly available and have demonstrated performance equal to or better than state-of-the-art LLMs of similar size [155], [158], [159]. In addition, the three LLM models manifest promising diverse task-handling abilities with an acceptable computational resource requirement. Hence, we consider these the best-fit subject LLMs to deliver representative results and insights.

Experimental Environment. All of our experiments were conducted on a server with a 4.5GHz AMD 5955WX 16-Core CPU, 256GB RAM, and two NVIDIA A6000 GPUs with 48GB VRAM each. The overall computational time takes more than 6,300 hours.

Hyperparameters Settings. As mentioned in Section III-B, our framework encloses various state abstraction and modeling methods with numerous hyperparameters. Therefore, there could be myriad possible hyperparameter combinations in our design that are not feasible to fully evaluate. Even though, to better understand the effectiveness of our framework, with our limited computational resources, we tried our best and conducted evaluations on as many as 180 representative hyperparameter settings to investigate the characteristics of different settings. The hyperparameters of our experiments are summarized in Table III.

Evaluation Metrics. Evaluation metrics are another crucial segment of our framework, as these metrics are devoted to presenting a transparent and comprehensive understanding of the quality of the constructed model as well as the effectiveness of the framework across different applications. Although some previous metrics are proposed by literature [54], [122], [127], [141], [143], [142], [144], [145], [146], only some of them are applicable to the context of LLMs. Thus, we carefully select 6 widely used metrics to assess the quality of the abstract model and propose another 6 metrics to evaluate the effectiveness of the model in terms of different trustworthiness perspectives. With these 12 metrics, we aim to conduct a relatively comprehensive evaluation as much as we can to obtain an in-depth understanding of our framework under different hyperparameter configurations, as well as the impact of hyperparameters on the effectiveness of LUNA. The metrics used for evaluation in this study are summarized in Table I and Table II.

2) *Subject Trustworthiness Perspective:* To better understand the effectiveness of the proposed framework across diverse tasks, we select a set of challenging and representative datasets. Mainly, as described in Section II-B, we assess the quality of the abstract model from three trustworthiness perspectives: Out-of-Distribution Detection, Adversarial Attack, and Hallucination. These three perspectives are widely observed and critical trustworthiness concerns of LLMs [9], [111], [138], [152], [160], [161]. Each of these has unique patterns (formats) that are capable of investigating both the effectiveness and the generality of our framework.

Out-of-Distribution Detection. We adapt the sentiment analysis dataset created by Wang et al. [9]. It is based on the SST-2 dataset [162] and contains word-level and sentence-level style transferred data, where the original sentences are transformed to another style. It contains a total of 9,603 sentences, with 873 in-distribution (ID) data and 8,730 OOD data.

Adversarial Attack. For the adversarial attack dataset, we use AdvGLUE++ [9], which consists of three types of tasks (sentiment classification, duplicate question detection, and multi-genre natural language inference) and five word-level attack methods. It contains 11,484 data in total.

Hallucination. For the hallucination perspective, we choose the following datasets

TruthfulQA [138], which is designed for measuring the *truthfulness* of LLM in generating answers to questions. It consists of 817 questions, with 38 categories of falsehood, e.g., misconceptions and fiction. The ground truth of the answers is judged by fine-tuned GPT-3 models [138] to classify each answer as true or false. We used two types of coding tasks, code generation and code summarization, to assess the reliability of the LLMs.

HumanEval [163] and MBPP [164] are selected to test the code generation ability. Both datasets measure the accuracy of Python programs generated from natural language docstrings, with 164 and 399 problems, respectively. We used the pass@1 [163] metric to evaluate code generation, which assesses the accuracy of LLM-generated code against test cases included in the datasets.

TL-CodeSum [165] and CodeSearchNet [166] are datasets to assess the code summarization capability. TL-CodeSum includes 87,136 pairs of methods and summaries that were scraped from 9,732 Java projects created between 2015 and 2016 on GitHub. Each project had at least 20 stars. We follow the previous work [167] to filter the duplicated instances in TL-CodeSum. CodeSearchNet is a source code dataset collected from open-source GitHub repositories. It includes code summarization data in six programming languages: Java, Go, JavaScript, PHP, Python, and Ruby. Both datasets were divided into training, validation, and testing sets in an 8:1:1 ratio. We extracted 5,000 instances from the training, validation, and test sets of TL-CodeSum. Similarly, for CodeSearchNet, we extracted 5,000 instances only from the Java subset's training, validation, and test sets. The ground truth of the answers is judged by GPT-3.5-Turbo-Instruct [168] to classify each answer as true or false.

Semantics Binding Across Different Perspectives. In hallucination detection, we bind semantics directly to states based on the *truthfulness* of the LLM's output answer [138], [169]. For TruthfulQA, the truthfulness is the output of a finetuned GPT-3-13B (GPT-judge) that is specified for estimating the degree of whether each answer is true or false, which is the common practice on TruthfulQA [138]. For code generation tasks such as HumanEval and MBPP, we measure truthfulness using the pass@1 [163], which evaluates the precision of code generation. An answer is considered true if it passes all test cases and false if any failure occurs. In code summarization tasks like CodeSearchNet-Java and TL-CodeSum, we use GPT-3.5-Turbo-Instruct [168] to determine the truthfulness of summarized content, classifying it as true or false. We instruct GPT-3.5 to evaluate the quality of the summary in comparison to the ground truth, with summaries that are accurate and relevant being marked as true and those that are inaccurate or irrelevant marked as false. In the OOD sample and adversarial attack

detection task, instead of state-level binding, we focus on the *transition probability* as the semantic representation.

C. RQ1: Can the Abstract Model Differentiate the Normal and Abnormal Behaviors of LLM?

Evaluation Design: Abnormal behavior awareness is a vital step for LLM analysis and interpretation. Therefore, RQ1 is devoted to conducting a preliminary investigation to acknowledge whether the constructed abstract models have the *potential* to characterize the behavior of the LLM from the lens of abnormality.

In particular, as mentioned in Section IV-A, our analysis centers on inspecting the distribution difference of the *transitions probabilities* from the abstract models between the normal and abnormal instances. We consider *transitions probabilities* as valid representatives of models' characteristics since they encapsulate the intrinsic and irreducible nature of state transition.

To answer RQ1, we assess the difference between normal and abnormal data *qualitatively* and *quantitatively* from three distributions: 1) the normal instances in the training dataset, 2) the normal instances in the test dataset, and 3) the abnormal instances in the test dataset. The normal instances in the training datasets and test datasets are considered to have similar distributions and represent the corpus that the LLM should properly process. In contrast, the abnormal instances in the test dataset are the contexts that are out of the scope of the training data. We consider the subject LLM to have abnormal behavior characteristics (e.g., faulty outputs, irregular hidden states behaviors) when processing these different instances. Furthermore, we expect to capture such dissimilarity in the abstract models from the lens of transition probabilities, which can reveal the consistency between the subject LLM and the corresponding abstract model. The hyperparameters of the abstract model are randomly picked from the hyperparameter space.

For the qualitative assessment, we rely on the Kernel Density Estimation (KDE) [170] plot to observe the distribution of transition probabilities between three types of instances. In terms of quantitative assessment, we investigate the statistical significance of three distributions using Mann-Whitney U test [171] (i.e., p-value).

Results:

We detail the distribution difference assessment from the KDE plot and the Mann-Whitney U test, respectively, and summarize the findings at the end of this subsection.

- **Qualitative Assessment.** Fig. 6 illustrates the distribution of transition probabilities w.r.t. three types of instances (normal instances in training data, normal instances in testing data, and abnormal instances in test data) across seven different tasks.

Among all seven tasks, the distributions of the transitions are highly aligned for normal instances in the training and test data. Namely, the abstract model has consistent behavior characteristics when dealing with normal instances. In terms of the normal and the abnormal instances, we also notice divergent distribution shapes in SST-2,

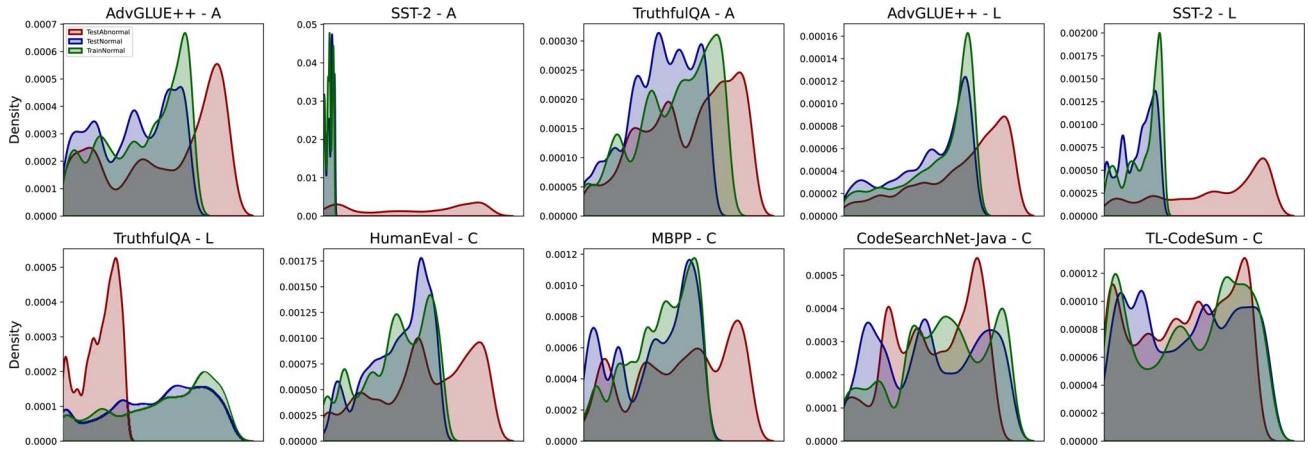


Fig. 6. RQ1: Distribution of transition probabilities of ten studied tasks.

AdvGLUE++, HumanEval, MBPP, and CodeSearchNet-Javadatasets. However, TL-CodeSum’s normal and abnormal instances have less difference by visually inspecting the distribution of transitions. This observation supports that normal and abnormal instances can be potentially distinguished by analyzing the distribution of transition probabilities of the abstract model. This ability to distinguish abnormal samples is seen in six out of seven tasks.

- **Quantitative Assessment.** We further conduct statistical significance tests to consolidate the visual findings from the KDE plots. As presented in Table IV, we show the p-value of the randomly picked model and the significant proportion, which represents the proportion of all models with different hyperparameter settings (180 hyperparameter settings in total) that resulted in a p-value less than 0.05. We find that abstract model features can be highly impacted by tasks. In hallucination, the proportion can be as low as 20% on TruthfulQA but as high as 64% on HumanEval. It indicates that although not all abstract models we built can detect abnormal instances, at least 20% of abstract models have the potential to detect abnormal instances.

Based on our findings, it can be shown that LUNA has the potential to detect abnormal behaviors, while its level of effectiveness may vary depending on the specific abstract model configurations used. The transition probabilities of the abstract model are aligned with normal instances and have significant differences while abnormal instances are encountered.

Answer to RQ1: Our experiment results confirm that the abstract model has the potential to characterize the anomalies of the subject LLM.

D. RQ2: How Do Different Modeling Techniques and Corresponding Configurations Impact the Quality of the Abstract Model?

As shown in RQ1, the abstract model is capable of detecting abnormal behavior of the subject LLM. In RQ2, we further examine the factors that impact the quality of the abstract models

TABLE IV

RQ1: STATISTICAL DIFFERENCES BETWEEN DISTRIBUTIONS OF NORMAL AND ABNORMAL AND SIGNIFICANT PROPORTION (THE PROPORTION OF SIGNIFICANT MODELS OVER ALL MODELS WITH DIVERSE HYPERPARAMETER SETTINGS). TQA: TRUTHFULQA, HE: HUMANEVAL, CSNJ: CODESEARCHNET-JAVA, TCS: TL-CODESUM. A: ALPACA-7B, L: LLAMA2-7B, C: CODELLAMA-13B-INSTRUCT

Perspective	LLM	p-value	Significant Proportion
OOD		1.02e-15	51%
Adversarial	A	6.37e-35	51%
Hallucination-TQA		4.82e-206	20%
OOD		1.75e-10	50%
Adversarial	L	1.09e-14	51%
Hallucination-TQA		3.32e-18	33%
Hallucination-HE		3.84e-11	64%
Hallucination-MBPP	C	2.33e-4	49%
Hallucination-CSNJ		3.69e-194	36%
Hallucination-TCS		4.72e-41	35%

from the state abstraction and trace construction perspectives. We first study the hyperparameters, including PCA dimensions, history steps, partition techniques (GMM, K-means, and Grid), and modeling methods (DTMC and HMM), as shown in Table III. Specifically, we aim to understand how these factors can benefit the general model analysis in terms of metrics, which include Succinctness (SUC), Stationary Distribution Entropy (SDE), Sink State (SS), Sensitivity (SEN), Coverage (COV), and Perplexity (PERP).

We normalized the metric values based on rank, indicating their relative ranking across metrics rather than absolute magnitudes. A higher normalized value means a better rank compared to other settings in the metric. Detailed metrics values for different PCA dimensions, cluster methods, and model types are available at our website <https://sites.google.com/view/llm-luna>.

1) RQ2.1: How Is the State Abstraction Correlated With Abstract Model-Wise Evaluation Metrics?:

Evaluation Design: As mentioned in Section III-B, we conduct a series of dimension reduction and state abstraction techniques to decompose and narrow down the large-scale concrete state

space of the LLM. Specifically, We apply PCA for dimension reduction on collected concrete states from the subject LLM. One crucial parameter of PCA is the number of components retained for the abstract state; therefore, we select three different levels of component numbers (Low, Medium, and High) to investigate how the degree of dimension reduction impacts the quality of the abstract model. It is worth mentioning that we set the PCA components to three comparative levels instead of concrete numbers due to the different state aggregation mechanisms performed by the three partition methods. Namely, a number of PCA components processable by GMM may not be feasible in terms of the Grid method; thus, for each state partition method, we arrange the PCA components to comparative levels to investigate its effect on the quality of the model. Specifically, as shown in Table III, we select {512, 1024, 2048} as corresponding low, medium, and high PCA component settings for cluster-based state partition method (KMeans and GMM), and {3, 5, 10} for grid-based method.

State partition techniques are subsequently applied to aggregate the concrete states with close spatial distance into one abstract state. We utilize three commonly used state partition approaches, e.g., GMM, KMeans, and Grid, to probe their effectiveness in mapping the large continuous concrete state space onto a compact discrete state space. To understand the potential impact of the different dimension settings from PCA, we conduct each state partition method on different PCA component settings and take the average performance across all PCA settings as the final result.

We investigate the impacts of PCA components and state partition methods, including cluster-based methods like GMM and KMeans, as well as the Grid method, in shaping the quality of abstract models through dimensionality reduction and state abstraction.

In terms of the abstract model quality measurement, we adopt the *abstract model-wise metrics* specified in Section III-D to inspect the characteristics of different abstraction settings from various aspects.

Results: From Figs. 7 and 8, we obtain the following findings about PCA dimensions and state abstraction approaches:

- **PCA Components:** Intuitively, Fig. 7 shows that an increase in the number of PCA components generally enhances both perplexity and coverage, with nine out of ten tasks exhibiting improved perplexity and eight out of ten tasks exhibiting better coverage. The perplexity measures the quality of the abstract model from the degree of well-fitting to the training distribution and the unpredictability within its transitions, respectively. Meanwhile, coverage measures the abstract model's ability to handle unseen states or transitions. This evaluation indicates the completeness of the model abstraction. Few unseen states in the test data show that the abstract model is powerful enough to handle the incoming data with the knowledge gained from the training data. Therefore, we consider a higher number of PCA components can reinforce the construction of the abstract model through distribution matching (perplexity) and the exploration of the abstract state space with limited training data (coverage).

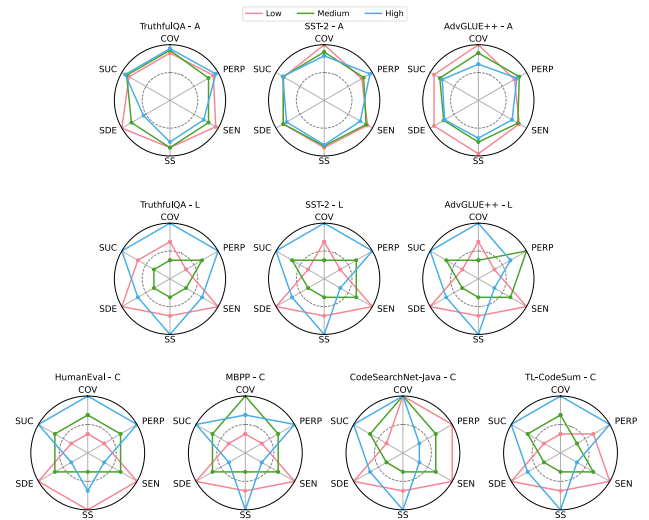


Fig. 7. RQ2.1: Model-wise metrics w.r.t. PCA dimension (number of components).

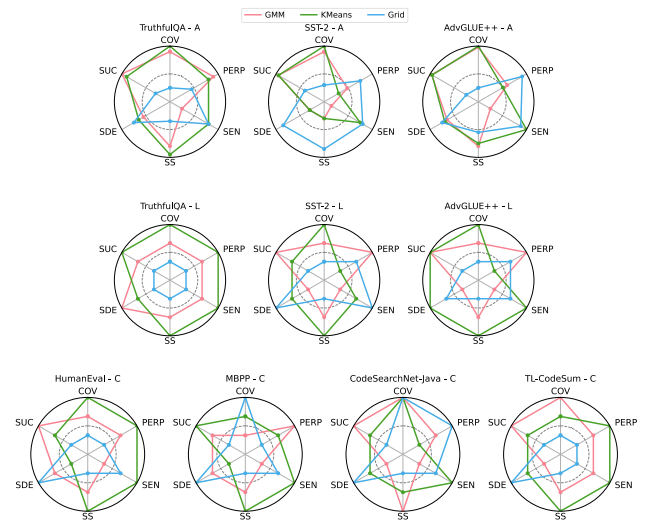


Fig. 8. RQ2.2: Model-wise metrics w.r.t. state partition method.

Nevertheless, the increase in the number of PCA dimensions adversely affects the sensitivity and the stationary distribution entropy. Sensitivity measures how the model reacts to small variations in concrete states' value and whether it changes the labels of abstract states. Stationary distribution entropy measures the divergence between the model's stationary distribution entropy and the average of its stochastic and stable bounds. This metric quantifies the degree to which the model's behavior regarding state transitions deviates from an equilibrium between predictability and randomness. Overly persevered state features (PCA components) increase the sensitivity and randomness of the semantics on abstract states. This means that the semantics become more easily influenced by small perturbations, leading to an unstable abstract model that

is susceptible to irrelevant information and noise. Producing a relatively large state space after the abstraction can prohibit the robustness of small perturbations (sensitivity) and find the right balance between stability and variability (stationary distribution entropy).

- **Cluster-based method:** Regardless of the PCA dimension, clustering-based methods, KMeans, and GMM usually present the highest or near-highest values regarding coverage and succinctness across ten tasks, as shown in Fig. 8. We consider the clustering-based state abstraction methods (KMeans and GMM) to be more efficient in aggregating the concrete states. Namely, unlike the grid-based approaches, the clustering-based methods only create new abstract states if a group of concrete states is gathered within certain spatial distances; thus, less abstract state space is generated for sparse concrete states. GMM's performance typically lies between KMeans and Grid. It displayed moderate mean values for most metrics, such as coverage and sink state. In addition, we also observe that KMeans achieves a higher score on the sensitivity metric than GMM. As mentioned in Section III-D, the sensitivity metric measures the change of abstract states against small perturbations on concrete states. Thus, the abstract states formed by GMM can retroactively signify the small perturbations that may drastically alter the outputs of the LLM, similar to the impact of high PCA dimensions.

Furthermore, we find that clustering-based methods have higher correlations compared to the grid-based method.

- **Grid-based method:** The grid-based approach performs better than cluster-based methods in terms of stationary distribution entropy (except for TruthfulQA) while having comparable scores on sensitivity and perplexity. Additionally, the grid-based method seems particularly adept for tasks related to OOD and Adversarial perspectives, as observed in datasets SST-2 and AdvGLUE++, where its application significantly surpasses that in tasks associated with Hallucination perspectives, such as TruthfulQA, HumanEval, MBPP, CodeSearchNet-Java, and TL-CodeSum. As highlighted in Section IV-B2, for OOD and Adversarial, we bind semantics with transitions. It suggests that the grid partition method potentially has advantages in imitating the distribution and transition characteristics of the subject LLM, especially on tasks with semantics bound on transitions.

Additionally, we notice a performance drop in coverage and succinctness, and we consider such limitations to be caused by the nature of the grid method. Namely, the grid-based method uniformly partitions the concrete state space along each dimension; therefore, an abstract state may be created even if no concrete states fall in this grid partition. If the dimension of the concrete space is high (depends on PCA) but the concrete states are densely distributed to certain areas, there will be a large portion of void abstract states generated (e.g., no abstract states and transitions exist in certain areas of abstract state space).

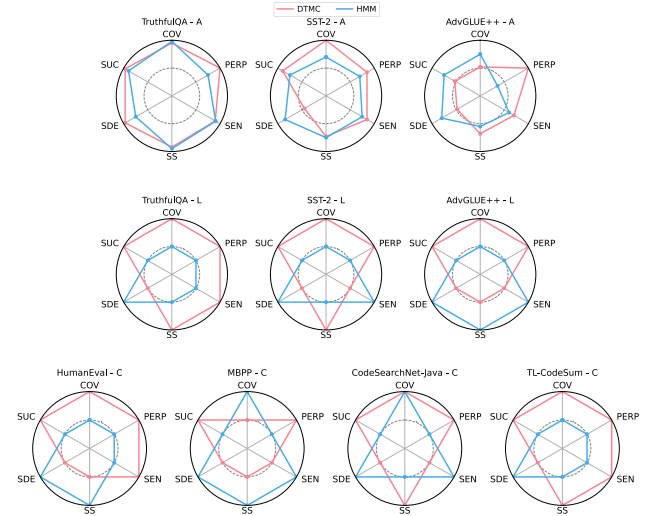


Fig. 9. RQ2.2: Model-wise metrics w.r.t. abstract model type.

Answer to RQ2.1: In the evaluation, a specific design on the number of PCA components is needed to balance the trade-offs among different quality metrics. The cluster-based method usually has advantages in state space reduction and exploration but falls short of preserving the distribution and deterministic nature of transitions. The grid-based method shows the opposite features.

2) RQ2.2: How Is the model Construction Method Correlated With Abstract Model-Wise Evaluation Metrics?:

Evaluation Design: Different types of model construction approaches are crucial regarding the quality of the abstract model as they concatenate the abstract states and reproduce the transition property of the subject LLM. In this RQ, we take DTMC and HMM as two candidate model construction methods to investigate their impact on the abstract model-wise properties.

Results: After analyzing the results in Fig. 9, we have several findings for model construction methods. Despite the number of PCA components and state abstraction techniques, DTMC shows close or beyond performance on succinctness, coverage, sensitivity, sink state, and perplexity on the majority of tasks. In terms of HMM, HMM generally lags behind DTMC in most metrics except stationary distribution entropy and sensitivity. We consider such differences to be attributable to the processes of building abstract transitions. Specifically, DTMC maps the abstract transitions directly according to the existence of concrete transitions, whereas HMM leverages a fitting algorithm to compute the transition probability with a maximized likelihood of observations.

The reason for HMM's relatively inadequate performance on model-wise metrics might be the two-level state abstraction mechanism, i.e., fit the hidden states and transitions on top of the abstract state.

Conversely, the direct transition abstraction of DTMC can be more effective in tracing back the transition characteristics of the LLM. Considering the specialties of LLMs, a desired model construction method should effectively and efficiently

capture and characterize the transition properties of the LLM for specific tasks.

Answer to RQ2.2: In our evaluated settings, the choice between DTMC and HMM largely hinges on the specific metric of interest. DTMC's versatile performance makes it a premier choice for downstream applications.

E. RQ3: How Does the Framework Perform Across Target Trustworthiness Perspectives, and How Is its Performance Correlated With Both Semantics-Wise and Abstract Model-Wise Metrics?

In this RQ, we first want to examine the *effectiveness* of our model-based analysis, i.e., whether it could detect abnormal behavior from the three studied trustworthiness perspectives.

Additionally, we are also interested in investigating the correlation between the newly proposed semantic-wise metrics and the analysis performance. Recall that semantics-wise metrics are designed to measure the quality of the abstract model in terms of *semantics*, which is supposed to have strong correlations with the analysis task performance. We want to confirm this point in this RQ. This exploration can also be used to guide the selection of a good abstract model for the analysis. Similarly, the correlation between abstract model-wise metrics and the performance is examined for the model selection procedure.

1) *RQ3.1: How Does Our Framework Perform on Trustworthiness Perspectives Regarding Semantics-Wise Metrics?:*
Evaluation Design: In RQ3.1, we first try to assess the effectiveness of the model-based analysis across three trustworthiness perspectives by checking the performance of the abnormal behavior detection (as described in Section III-E).

We choose ROC AUC [154], as the metric to evaluate the performance of the detection task. Typically, if the ROC AUC of a method is higher than 0.5, we consider it an effective one (better than random). We check the performance of detection based on models with different hyperparameter settings (a total of 180 settings) and show their performance (mean, maximum, minimum, median, standard deviation, and variation). By initially examining the ROC AUC, we gain a preliminary understanding of the effectiveness of our model-based analysis over the hyperparameter space.

Moreover, we also examine the correlation between the newly introduced semantic-wise metrics and ROC AUC by computing the Pearson [172] and Kendall's tau (Kendall) [173] correlation coefficients. Pearson correlation is a widely used parametric measurement but can be potentially affected by outliers. In contrast, Kendall is a non-parametric measurement that is considered more robust against outliers as it relies on the ordinal ranks of the data instead of the actual values [174]. Including both Kendall and Pearson coefficients can initiate a more robust correlation measurement for our study.

To be more specific, we go over all hyperparameter settings, and for each specific hyperparameter setting, we generate a corresponding abstract model, determine its ROC AUC for every trustworthiness perspective, and calculate the semantic-wise metrics for these settings. Each ROC AUC is correlated

TABLE V
 RQ3.1: THE ROC AUC FOR EACH PERSPECTIVE. MAX: MAXIMUM, MIN: MINIMUM, STD. DEV.: STANDARD DEVIATION, VAR.: VARIANCE. TQA: TRUTHFULQA, HE: HUMANEVAL, CSNJ: CODESEARCHNET-JAVA, TCS: TL-CODESUM. A: ALPACA-7B, L: LLAMA2-7B, C: CODELLAMA-13B-INSTRUCT

Perspectives	LLM	Mean	Max	Min	Median	Std. Dev.	Var.
OOD	A	0.55	0.65	0.50	0.53	0.04	0.00
Adversarial		0.59	0.83	0.50	0.56	0.09	0.01
Hallucination-TQA		0.67	0.71	0.55	0.69	0.04	0.00
OOD	L	0.63	0.85	0.50	0.64	0.11	0.01
Adversarial		0.62	0.80	0.50	0.63	0.09	0.01
Hallucination-TQA		0.64	0.69	0.52	0.64	0.04	0.00
Hallucination-HE	C	0.69	0.90	0.52	0.69	0.04	0.00
Hallucination-MBPP		0.67	0.73	0.54	0.67	0.05	0.00
Hallucination-CSNJ		0.66	0.70	0.53	0.66	0.03	0.00
Hallucination-TCS		0.70	0.74	0.62	0.72	0.03	0.00

with the corresponding semantic-wise metrics, which can be computed by the Pearson coefficient between the ROC AUC and the values of each semantic-wise metric. Recall that semantics represent the level of satisfaction of the LLM w.r.t. the trustworthiness perspective. This investigation helps us understand whether the newly proposed semantics-wise metrics have the potential to indicate the performance of the analysis procedure to some extent. Recall that the semantics-wise metrics include Preciseness (PRE), Entropy (ENT), Surprise Level (SL), *n*-gram Derivative Trend (NDT), Instance Value Trend (IVT), and *n*-gram Value Trend (NVT).

Results: Table V shows the statistics of ROC AUC across a significant subset of all abstract models, as indicated by a p-value less than 0.05. The performance of detecting OOD samples (SST-2) on Alpaca-7b is worse than the other tasks, with the lowest mean (0.55), maximum (0.65), and median (0.53) ROC AUC, which is reasonable, as the OOD samples only differ very slightly from the original one and are hard to be detected. The same task works better on Llama2-7b. Although the relatively high standard deviation (0.11) indicates considerable variability in the results, the mean (0.63) and maximum (0.85) both show that certain model configurations yield higher effectiveness in OOD sample detection.

Moreover, the maximum performance of HumanEval achieves the highest results among all tasks (0.90). This conforms to the result of RQ1 that the significant proportion of models built on HumanEval is higher than the others, indicating that HumanEval is intrinsically easier for abnormal behavior detection, and our model indeed can catch the difference between normal and abnormal behavior. We can also see that the data is concentrated, with almost zero variance and a small standard deviation (less than 0.1 except SST-2 on Llama2-7b).

Table VI shows the experimental result of the top-5 performance with their hyperparameters setting of the detection tasks. We can see that all top-5 models are DTMC. The reason for the unsatisfying performance of HMM might be the two-layer state abstraction, i.e., HMM learns the hidden states on the abstract states.

Additionally, the Grid method appears more suitable for OOD and Adversarial perspectives, where it significantly

TABLE VI
RQ3.1: TOP 5 SETTINGS FOR EACH TASK AND ROC AUC.
TQA: TRUTHFULQA, HE: HUMANEVAL, CSNJ: CODESEARCHNET-JAVA,
TCS: TL-CODESUM, A: ALPACA-7B, L: LLAMA2-7B,
C: CODELLAMA-13B-INSTRUCT

Perspective	LLM	PCA	Partition	#State	Model	ROC AUC
OOD	A	1024	GMM	200	DTMC	0.65
		512	GMM	512	DTMC	0.64
		5	Grid	1.0e+5	DTMC	0.63
		10	Grid	1.0e+10	DTMC	0.62
		512	GMM	600	DTMC	0.62
	L	2048	GMM	200	DTMC	0.85
		5	Grid	7.6e+5	DTMC	0.83
		3	Grid	3375	DTMC	0.83
		2048	GMM	400	DTMC	0.83
		1024	GMM	200	DTMC	0.82
Adversarial	A	10	Grid	1.5e+11	DTMC	0.83
		10	Grid	1.0e+10	DTMC	0.80
		10	Grid	1.0e+10	DTMC	0.79
		10	Grid	1.0e+10	DTMC	0.76
		5	Grid	1.5e+5	DTMC	0.75
	L	5	Grid	7.6e+5	DTMC	0.80
		3	Grid	3375	DTMC	0.72
		5	Grid	7.6e+5	DTMC	0.72
		5	Grid	7.6e+5	DTMC	0.71
		5	Grid	1.0e+5	DTMC	0.70
Hallucination-TQA	A	3	Grid	125	DTMC	0.71
		3	Grid	1000	DTMC	0.71
		1024	GMM	600	DTMC	0.71
		2048	KMeans	400	DTMC	0.71
		2048	GMM	600	DTMC	0.70
	L	1024	KMeans	400	DTMC	0.69
		1024	KMeans	600	DTMC	0.69
		1024	GMM	600	DTMC	0.69
		512	KMeans	600	DTMC	0.69
		2048	GMM	400	DTMC	0.68
Hallucination-HE	C	2048	KMeans	600	DTMC	0.90
		512	GMM	600	DTMC	0.90
		1024	KMeans	400	DTMC	0.89
		512	KMeans	200	DTMC	0.89
		512	KMeans	400	DTMC	0.88
Hallucination-MBPP	C	5	Grid	1.0e+5	DTMC	0.73
		512	GMM	512	DTMC	0.73
		512	KMeans	512	DTMC	0.72
		5	Grid	7.6e+5	DTMC	0.72
		1024	GMM	600	DTMC	0.72
Hallucination-CSNJ	C	2048	GMM	200	DTMC	0.70
		512	GMM	200	DTMC	0.69
		1024	GMM	400	DTMC	0.69
		2048	GMM	400	DTMC	0.69
		1024	GMM	200	DTMC	0.69
Hallucination-TCS	C	1024	KMeans	200	DTMC	0.74
		1024	KMeans	512	DTMC	0.74
		5	Grid	1.0e+5	DTMC	0.74
		512	KMeans	600	DTMC	0.74
		2048	KMeans	600	DTMC	0.74

outperformed compared to its application in hallucination tasks across datasets like TruthfulQA, HumanEval, MBPP, CodeSearchNet-Java, and TL-CodeSum. As discussed in Section IV-B2, for OOD and Adversarial contexts, we bind semantics with transitions. This result is echoed in the findings of RQ2.1, indicating that tasks involving transition-binding semantics show higher perplexity compared to those focused on state-binding.

We conclude that our model-based abnormal behavior detection is effective, as it is higher than the random approach (with ROC AUC 0.5).

For the correlation analysis, the Pearson and Kendall correlation coefficients for each semantic-wise metric w.r.t. ROC AUC values for different abstract model settings are presented in Table VII. Each cell displays the Pearson

TABLE VII
RQ3.1: PEARSON AND KENDALL COEFFICIENTS OF SEMANTIC-WISE METRICS W.R.T. ROC AUC. PEARSON VALUE FOLLOWED BY THE KENDALL VALUE IN PARENTHESES, FORMATTED AS *PEARSON(KENDALL)*.
ADV: ADVERSARIAL, HAL: HALLUCINATION, TQA: TRUTHFULQA, HE: HUMANEVAL, CSNJ: CODESEARCHNET-JAVA, TCS: TL-CODESUM, A: ALPACA-7B, L: LLAMA2-7B, C: CODELLAMA-13B-INSTRUCT

Perspective	LLM	PRE	ENT	SL	NDT	IVT	NVT
OOD	A	0.94 (0.72)	0.34 (0.25)	0.14 (0.01)	-0.25 (-0.10)	-0.25 (-0.16)	-0.34 (-0.13)
ADV		0.95 (0.79)	-0.59 (-0.21)	-0.18 (-1.23)	0.48 (0.14)	-0.19 (0.08)	-0.75 (-0.74)
HAL-TQA		0.81 (0.75)	-0.07 (0.11)	-0.18 (-0.16)	-0.38 (-0.37)	-0.03 (-0.13)	0.51 (0.16)
OOD	L	0.92 (0.73)	0.79 (0.44)	0.88 (0.65)	-0.65 (-0.52)	-0.57 (-0.19)	-0.52 (-0.15)
ADV		0.98 (0.71)	0.47 (0.15)	-0.23 (-0.04)	-0.52 (-0.12)	-0.16 (0.17)	0.60 (0.17)
HAL-TQA		1.00 (1.00)	0.52 (0.37)	0.69 (0.51)	-0.30 (-0.28)	-0.46 (-0.23)	-0.36 (0.03)
HAL-HE	C	1.00 (1.00)	0.77 (0.46)	0.63 (0.24)	-0.23 (-0.21)	-0.49 (-0.21)	-0.47 (-0.23)
HAL-MBPP		0.99 (0.83)	0.97 (0.50)	0.78 (0.46)	-0.21 (-0.24)	-0.35 (-0.14)	-0.50 (-0.50)
HAL-CSNJ		0.96 (0.74)	0.58 (0.41)	0.61 (0.30)	-0.22 (-0.20)	-0.46 (-0.21)	-0.42 (-0.08)
HAL-TCS		0.95 (0.71)	0.79 (0.42)	0.83 (0.30)	-0.23 (-0.27)	-0.46 (-0.22)	-0.67 (-0.32)

value followed by the Kendall value in parentheses, formatted as *Pearson(Kendall)*. Overall, we have the following observations.

Generally, Kendall values are slightly smaller than Pearson values. This is potentially because Kendall can be more robust to outliers than Pearson. Additionally, we observed a consistent pattern between Pearson and Kendall coefficients across most semantic-wise metrics, which further confirms the robustness of the results. Moreover, Semantics preciseness (PRE) and ROC AUC have a strong positive correlation, which suggests the necessity of building a semantically precise for getting good analysis performance. Furthermore, in the case of four coding tasks using the same LLM, CodeLlama-13b-Instruct, there is a consistent correlation across all six metrics. However, for the Alpaca-7b and Llama2-7b datasets, we observe a mix of positive and negative correlations among metrics such as surprise level (SL) and n -gram value trend (NVT). This suggests that in practical applications, users should analyze different metrics accordingly for selecting appropriate hyperparameters, as they might show diverse correlations to the final analysis task.

Answer to RQ3.1: Our model-based abnormal behavior detection is effective in three trustworthiness perspectives. The semantics-wise metrics also show different correlations with the analysis performance in different tasks, except for preciseness.

2) RQ3.2: How Is the Performance of the Framework Correlated With the Abstract Model-Wise Metrics?:

Evaluation Design: In RQ3.2, we examine the correlation between the abstract model-wise metrics and ROC AUC by computing the Pearson [172] and Kendall [173] correlation coefficients between them. Our aim is to understand the correlation between the performance (Table VIII) and their corresponding model-wise metrics. Identifying this correlation can help us choose the abstract model with potentially good performance based on abstract model-wise metrics in the future.

Results: The correlation is shown in Table VIII. After analyzing the correlations between ROC AUC and abstract model-wise metrics, we have several findings.

Firstly, we observed similar patterns for model-wise metrics, as seen with semantic-wise metrics in RQ 3.1, where Kendall

TABLE VIII

RQ3.2: PEARSON AND KENDALL COEFFICIENTS OF MODEL-WISE METRICS W.R.T. ROC AUC FOR DIFFERENT TASKS. PEARSON VALUE FOLLOWED BY THE KENDALL VALUE IN PARENTHESES, FORMATTED AS $PEARSON(KENDALL)$.
 ADV: ADVERSARIAL, HAL: HALLUCINATION, TQA: TRUTHFULQA,
 HE: HUMAN EVAL, CSNJ: CODESEARCHNET-JAVA, TCS: TL-CODESUM.
 A: ALPACA-7B, L: LLAMA2-7B, C: CODELLAMA-13B-INSTRUCT

Perspective	LLM	SUC	COV	SEN	SS	PERP	SDE
OOD		-0.35 (-0.2)	-0.33 (-0.17)	0.05 (0.02)	0.35 (0.07)	0.46 (0.32)	0.27 (0.18)
ADV	A	-0.38 (-0.14)	-0.33 (-0.27)	0.04 (0.16)	0.34 (0.13)	0.46 (0.29)	0.27 (0.11)
HAL-TQA		0.02 (0.12)	0.06 (0.12)	0.20 (0.03)	0.04 (-0.02)	-0.08 (-0.05)	-0.44 (-0.11)
OOD		-0.05 (0.06)	-0.06 (-0.01)	-0.09 (-0.05)	-0.20 (-0.05)	0.90 (0.58)	-0.15 (-0.13)
ADV	L	-0.07 (0.01)	-0.28 (-0.38)	0.07 (0.06)	0.64 (0.32)	0.90 (0.72)	0.56 (0.46)
HAL-TQA		0.45 (0.29)	0.37 (0.43)	0.11 (0.12)	0.28 (0.32)	0.63 (0.59)	0.11 (0.01)
HAL-HE		0.30 (0.22)	0.33 (0.29)	0.06 (0.01)	-0.09 (-0.1)	0.57 (0.35)	-0.28 (-0.18)
HAL-MBPP	C	0.48 (0.17)	-0.11 (-0.08)	0.08 (-0.02)	-0.13 (-0.04)	0.56 (0.48)	-0.07 (-0.08)
HAL-CSNJ		0.35 (0.30)	-0.02 (-0.01)	0.05 (0.03)	0.16 (0.29)	0.37 (0.37)	-0.17 (-0.15)
HAL-TCS		0.47 (0.35)	0.42 (0.33)	0.17 (0.04)	0.30 (0.27)	0.61 (0.53)	-0.06 (-0.18)

values were slightly smaller than Pearson values but maintained consistent trends and patterns. This similarity further confirms the reliability of our findings. Moreover, there is a relatively higher positive correlation on both coefficients between perplexity (PERP) and ROC AUC, with the exception of TruthfulQA on Alpaca-7b. This suggests that with abstract models, which effectively mimic the distribution and transition traits of the target LLM, *LUNA* is likely to achieve superior ROC AUC scores. Besides, similar to the finding in RQ3.1, some metrics and ROC AUC exhibit different correlations with different trustworthiness perspectives. For example, succinctness (SUC) and coverage (COV) have mixed positive and negative correlations with the studied perspectives.

This difference highlights the importance of considering diverse metrics as performance indicators for a comprehensive assessment.

Answer to RQ3.2: Similar to semantics-wise metrics, abstract model-wise metrics exhibit different correlations with the analysis performance in different domains as well. In practice, the user can consider perplexity as a broadly applicable metric across various perspectives and tasks. Nonetheless, to achieve a thorough analysis performance, the user could use diverse combinations of metrics to guide their model selection process.

V. DISCUSSION

A. Comparative Analysis of Trustworthiness Assessment Techniques

Recent works SAPLMA by Azaria et al. [81] and Inference-Time Intervention (ITI) by Li et al. [169] have made notable contributions to enhance LLM's trustworthiness, each presenting unique approaches to assessing and improving the truthfulness of LLM outputs. This discussion seeks to compare these methods with *LUNA*, highlighting each approach's strengths, limitations, and applicability in various contexts. *LUNA* utilizes abstract model construction and stateful behavior analysis to assess the trustworthiness of LLMs comprehensively. This contrasts with SAPLMA, which focuses on using LLMs' hidden layer activations for content truthfulness classification, and ITI, which manipulates attention mechanisms

for truthfulness improvements. While SAPLMA and ITI offer insightful approaches for assessing truthfulness in natural language question-answering tasks, their focus does not extend to evaluating adversarial robustness or OOD perspectives, nor do they explore coding tasks. Specifically, SAPLMA targets tasks that can be straightforwardly categorized with binary true or false labels, limiting its application to more nuanced or complex scenarios. In contrast, *LUNA* is task-universal and more flexible with different trustworthiness perspectives. Meanwhile, ITI distinguishes itself by enhancing the truthfulness of outputs through shifting activations to the truthful direction during inference. Although *LUNA* possesses the potential for similar enhancements, exploring this capability in detail exceeds the current paper's scope and is designated for future investigation. This comparison demonstrates the challenge of directly comparing *LUNA* to SAPLMA and ITI. *LUNA*'s approach allows for a broader assessment of trustworthiness across various dimensions beyond truthfulness in natural language question answering.

Besides, *LUNA* is capable of conducting online monitoring and word-by-word inspection during the inference process due to the model-based analysis. The abstract models can be constructed at any token, along with its hidden information, during the inference process. However, both SAPLMA and ITI analyze the entire output's hidden information after the inference process has concluded, which limits their ability to perform real-time monitoring.

In summary, *LUNA* showcases its potential through a wide-ranging and flexible evaluation of LLM trustworthiness, surpassing SAPLMA and ITI.

B. Abstract Model Construction and Its Implications

Some research works have demonstrated that a well-constructed abstract model can behave as an indicator to reveal the internal behavior of the target neural network model [29], [54], [57], [122], [127]. Adequate model construction techniques are vital to retroactively reflect the corresponding characteristics of the studied system. Nevertheless, considering the very large model size and the distinct self-attention mechanism of LLMs, it is still unclear to what extent existing methods are effective on LLMs. Hence, our framework, *LUNA*, collaborates three state abstraction methods and two model construction techniques with a total of 180 different parameter configurations to extensively explore the effectiveness of popular model-based analysis approaches.

From the evaluation results, we find that cluster-based state partition methods (KMeans, GMM) and the grid-based method have distinct advantages on different model quality measurement metrics. Meanwhile, in terms of the methods of model construction, DTMC exhibited close or beyond performance on most of the abstract model quality metrics and ROC AUC¹ than HMM, which implies it is a potential candidate to model the state transition features of LLMs. It is worth noting that the efficacy of the abstraction and modeling techniques varies on

¹ Refer to Appendix B (available online) for details.

tasks and trustworthiness perspectives. For instance, KMeans gets superior performance scores on Succinctness and Coverage on both TruthfulQA and SST-2 datasets but relatively inadequate performance on AdvGLUE++ dataset. Such a finding signifies that explicit selection of methods and appropriate parameter tuning are necessary to maximize the effectiveness of existing techniques regarding abstract model construction. Therefore, advanced and LLM-specific abstract model construction techniques are called to capture and represent the behavior characteristics of LLMs regardless of types of tasks and trustworthiness perspectives.

C. Abstract Model Quality Measurement

In this work, we tried our best and chose as many as 12 metrics to initiate a relatively comprehensive understanding of the quality of the constructed model from both abstract model-wise and semantics-wise. Particularly, abstract model-wise metrics assess the intrinsic properties of the model regardless of subject trustworthiness perspectives, such as the stability of the model and the degree of well-fitting to the distribution of training data. We notice that Coverage and Succinctness, which measure the level of compression of the abstract model, provide more insights for dimension reduction and abstract state partition. Moreover, Stationary Distribution Entropy, Perplexity, and Sink State make more efforts to guide the selection of model construction methods and subsequent parameter tuning. Such metrics help to enhance the quality of the model towards better training distribution fitting and the ability against small perturbations.

In contrast, semantics-wise metrics measure the quality of the model from the angle of the degree of satisfaction w.r.t. trustworthiness perspectives. In particular, from Section IV-E, we notice that Preciseness, Entropy, and n -gram Value Trend are more correlated with the performance of the model regarding different trustworthiness perspectives. Some metrics may have varying degrees of adaptability on certain applications. For example, Surprise Level and n -gram Derivative Trend have finer effectiveness in describing the quality of the model on adversarial and hallucination detection.

In general, different metrics are needed to collaboratively guide the construction of the abstract model and secure the quality from diverse aspects. Also, some metrics are potentially fit to tackle specific downstream tasks or trustworthiness perspectives; thus, more research is called to prospect the explicit metrics for particular applications or quality requirements.

D. Model-Based Quality Assurance for LLM

The fast-growing popularity of LLMs highlights the escalating influence of LLMs across academia and industry [5], [7], [175]. With the witness to the adoption of LLMs in a large spectrum of practical applications, LLMs are expected to carry as foundation models to boost the software development lifecycle in which trustworthiness is critical. Namely, quality assurance techniques explicitly in the context of LLMs are urgently needed to enable the deployment of LLMs in more safety, reliability, security, and privacy-related applications.

Our framework *LUNA* aims to provide a general and versatile platform that assembles various modeling methods, downstream tasks, and trustworthiness perspectives to safeguard the quality of LLMs. Moreover, considering the extensibility of the framework, *LUNA* is expected to behave as a foundation that enables the following research to implement new advanced techniques for more diverse tasks across different domains. The results from Section IV confirm that the abstract model can act as a beacon to disclose abnormalities in the LLM when generating responses to different inputs. Specifically, the abstract model extracts and inspects the inner behavior of the LLM to detect whether it is under unintended conditions that can possibly produce nonfactual or erroneous outputs. The model embeds semantics w.r.t. different trustworthiness perspectives to extend its capability to tackle diverse quality concerns. In addition, we consider our framework can play roles in extensive quality assurance directions, such as online monitoring [176], [177], [178], [179], fault localization [180], [181], testing case generation [19], [21], [182], [183] and output repair [28], [184], [185]. For instance, by leveraging the trajectories of the states and corresponding semantics w.r.t. a specific output, it is possible to trace back and precisely localize the faulty segments within the output tokens.

E. Guidance and Insights for Practitioners

In this section, we use hallucination as a practical example to illustrate how practitioners can apply specific metrics to determine suitable settings for abstract models. Recognizing that this direction (model-based analysis of LLMs) is still in its early stages, with best practices yet to be fully shaped, we tried to provide some guidance and insights for the practitioners regarding the metric and configuration settings based on our empirical research on metric selection and configuration strategies in this paper.

As highlighted in Section III-A, the ability to create an abstract model that capably mimics the behavior of original LLMs is critical for the trustworthiness analysis conducted by *LUNA*. However, the challenge arises with *LUNA*'s extensive range of potential configurations (180 in our experimental framework), potentially making it difficult for practitioners to identify the most effective configuration. Based on our empirical evaluation and findings, we find a way that holds the potential to be helpful in identifying the appropriate setting, which involves assessing configurations against a selected set of metrics comprising 6 model-wise and 6 semantic-wise metrics.

Early observations from RQ3.1 and RQ3.2 reveal patterns that may assist in the selection of metrics crucial for evaluating the quality of abstract models. For instance, in the context of the hallucination, metrics such as PRE, ENT, SUC, and PERP show strong and consistent correlations to ROC AUC, as detailed in Tables VIII and VII.

Table IX shows the top-ranked configurations for each task from a hallucination perspective, determined by aggregating values of selected metrics. The results that indicate those findings are summarized in Table VI, which shows the potential of our approach in identifying appropriate settings, as all of the

TABLE IX
IDENTIFIED ABSTRACT MODEL CONFIGURATIONS FOR TASKS FROM
HALLUCINATION PERSPECTIVE. TQA: TRUTHFULQA, HE: HUMAN EVAL,
CSNJ: CODESEARCHNET-JAVA, TCS: TL-CODESUM. A: ALPACA-7B,
L: LLAMA2-7B, C: CODELLAMA-13B-INSTRUCT

Perspectives	LLM	PCA	Partition	#State	Model	ROC AUC
OOD	A	10	Grid	1.0e+10	DTMC	0.62
OOD	L	2048	GMM	200	DTMC	0.85
Adversarial	A	5	Grid	1.5e+5	DTMC	0.75
Adversarial	L	5	Grid	7.6e+5	DTMC	0.85
Hallucination-TQA	A	3	Grid	1000	DTMC	0.71
Hallucination-TQA	L	3	Grid	125	DTMC	0.65
Hallucination-HE		2048	KMeans	600	DTMC	0.90
Hallucination-MBPP		5	Grid	1.0e+5	DTMC	0.73
Hallucination-CSNJ	C	2048	GMM	200	DTMC	0.70
Hallucination-TCS		5	Grid	1.0e+5	DTMC	0.74

four coding tasks — HumanEval, MBPP, CodeSearchNet-Java, and TL-CodeSum— achieve maximum ROC AUC. While the performance for TruthfulQA on Llama2-7b slightly fell short of the maximum, it still ranks to be competitive, within the top 5 configurations, suggesting the preliminary insights of the selected metrics in guiding abstract model setting selection.

In summary, practitioners are recommended to first identify metrics that exhibit strong and consistent correlations with ROC AUC. Following this step, practitioners could then rank the various configurations based on the chosen metrics to identify the configuration that presents the most potentially beneficial metric outcomes. For instance, in the context of hallucination analysis, metrics such as PRE, ENT, SUC, and PERP are highly ranked and recommended, with their effectiveness across code generation, code summarization, and QA datasets illustrated in Table IX.

The metric and configuration identification process in this section is mostly based on our empirical evaluation steps and results. Although there might be more advanced metric and configuration selection strategies that could be explored by the practitioners to further enhance the analysis performance, in this paper, our discussion aims to provide practitioners with early observations on how using selected metrics can be helpful to the identification of well-performed abstract model configurations, thereby improving *LUNA*'s effectiveness in quality assurance.

In this paper, we take an early step to present a model-based LLM analysis framework, *LUNA*, to initiate exploratory research towards the quality assurance of LLMs. Our experiment results show that the abstract model can capture the abnormal behaviors of the LLM from its hidden state information. We conduct a series of modeling techniques with a diverse set of quality measurement metrics to deliver a comprehensive understanding of the capability and effectiveness of our framework. Hence, we find that *LUNA* shows performant abilities to detect the suspicious generations of LLMs w.r.t. different trustworthiness perspectives.

VI. THREATS TO VALIDITY

In this Section, we discuss the threats that may affect the validity of our study and the actions we have taken to mitigate them.

Internal Threats. The configurations in state abstraction and model construction can be an internal threat that impacts our evaluation results. A satisfactory abstract model should, on the one side, maximally narrow down the concrete state space to make it more compact and processable; on the other side, form abstract states that are representative of distinct LLM behaviors. To mitigate this threat, in this study, we propose a total of 180 configuration settings that may affect the performance of the abstract model to obtain a comprehensive and constructive understanding of how different parameter configurations impact the effectiveness of the abstract model.

External Threats. The generality of our framework to other LLMs, tasks, and trustworthiness perspectives beyond this study can be an external threat. In light of different LLM structures, output types, and task requirements, our results may not always be applicable to other scenarios. To mitigate this threat, we select the three currently widely concerned trustworthiness perspectives on three different datasets to conduct the experiments. Likewise, multiple modeling techniques and evaluation metrics are enclosed in our framework to enhance its applicability to other trustworthiness perspectives and applications.

Construction Threats. It is possible that our evaluation metrics may not fully characterize all possible performance aspects of the model. To mitigate this threat, we investigate a large number of metrics of model quality measurements from previous works [54], [122], [127], [141], [142], and carefully select twelve different metrics from two categories: abstract model-wise and semantics-wise. The former measures the quality of the model from the angles of the level of abstraction, distribution fit, sensitivity, etc; the latter evaluates the model performance based on the level of satisfaction w.r.t. different trustworthiness. By incorporating these metrics, we tried our best to deliver an adequate assessment. Moreover, another possible threat is that metrics might not actually measure what is wanted. To counter this threat, we carefully check the implementation and evaluation of the metrics to ensure correct implementation. Additionally, as shown in the experimental evaluation (RQ2 and RQ3), we compute the Pearson coefficient between the performance of abnormal behavior detection and metrics value, and the correlation between the abstraction method and metrics. The results show that the metrics have different correlations with the analysis performance in diverse tasks. In this way, the model evaluation metrics are demonstrated to be able to assess the usefulness of the constructed model in the subsequent analysis procedure.

VII. RELATED WORK

A. Quality Assurance of LLM

Quality assurance in deep learning-driven NLP software has recently garnered significant interest from industry and academia. On one side, related research seeks to empirically evaluate the trustworthiness of these models more thoroughly and comprehensively. Meanwhile, there is a concerted demand and push toward devising advanced techniques to predict failures, identify ethical concerns, and enhance various abilities of current models.

Regarding empirical evaluation, some benchmarks have been proposed, addressing factual consistency [138], [152], [186], [187], robustness [111], [188], toxicity [189], and hallucination [152], [161] in tasks like QA and text summarization. These benchmarks comprise datasets that are either human-labeled [152], [187], extracted from external resources [161], [186], transformed from other datasets [111], [188], or labeled/generated by AI models [138], [189]. While many studies target specific AI model facets for select tasks, the multifaceted nature of LLMs warrants broader evaluations. Recent research delves into multiple capabilities of LLMs, encompassing faithfulness of QA [190], security of generated code [191] and its correctness [192], mathematical capabilities [193], and logical reasoning skills [194]. Notably, HELM [195] stands out as an important study in this domain. It conducts extensive tests across seven metrics in 42 scenarios for 30 language models, offering a comprehensive insight into the current landscape of LLMs. DecodingTrust [9] is another important benchmark assessment of LLMs that concentrates on diverse perspectives of trustworthiness. In our work, we select two important salient tasks from this study: adversarial detection and OOD detection.

These empirical studies reveal that while LLMs excel in various tasks, they often lack trustworthiness and transparency. To tackle these shortcomings, some recent studies suggest some promising directions such as data-centric methods [196], [197], [198], [199], uncertainty estimation [147], [200], [201], [202], [203], [204], [205], controlled decoding [206], [207], [208], [209], self-refinement [210], [211], [212], [213], [214], and leveraging external knowledge during inference [175], [215], [216], [217], [218], [219].

Data-centric approaches are model-agnostic and formulate related problems as unintended behavior detection. Typically, these methods gather data and train classifiers to identify undesired content. A notable instance is OpenAI’s moderation system, offered as an API service [198]. This system’s training data encompasses content related to sexuality, hate, violence, self-harm, and harassment. Uncertainty estimation, often lightweight and black-box in nature, uses uncertainty scores as indicators for the models’ trustworthiness. Manakul et al. [147], for instance, introduce a black-box hallucination detection technique based on token-level prediction likelihood and entropy, while Huang et al. [202] explore the efficacy of both single and multi-inference uncertainty estimation methods.

While the above two approaches focus more on detection, the remaining three aim to directly improve the generated content. Controlled decoding techniques freeze the base LLM while guiding the text generation to achieve the desired attributes. Mireshghallah et al. [208], for example, propose energy-based models to steer the distribution of generated text toward desired attributes, such as unbiased content. Cao et al. [209] suggest employing dead-end analysis to reduce LLM toxicity. Drawing inspiration from human introspection, self-refinement methods have been introduced. Huang et al. [212] instruct LLMs to generate confident answers for unlabeled questions, which are then used in further training. Madaan et al. [213] suggest that LLMs critique and refine their own outputs. Lastly, LLMs

augmented with external databases can address the “brain-in-the-vat” dilemma [220], leading to more accurate inferences. Examples include WIKI-based chatbots [217] and Retrieval-Augmented LLMs [218].

Among the relevant studies, the work by Azaria et al. [81] and Li et al. [169] bear the closest resemblance to ours. While the majority of these approaches adopt black-box methodologies, they try to analyze the relationship between LLMs’ internal and their trustworthiness. Azaria et al. utilize the hidden layer activations of LLMs as features to train a classifier for assessing the truthfulness of generated content. Li et al. first probe LLMs to find the correlation between truthfulness and attention heads and subsequently leverage this insight for inference-time intervention, aiming to produce more accurate responses. In contrast, our framework emphasizes holistic model extraction and stateful analysis, offering a more systematic exploration of the stateful characteristics inherent to LLMs.

B. Model-Based Analysis for Stateful DNNs

Interpreting the behavior of stateful deep neural networks is challenging, considering the potentially countless concrete states the model can reach and its near black-box nature. Fortunately, there have already been some successful attempts for the RNN-series, a representative stateful architecture before the transformer era. Some theoretical research indicates that, while RNNs are Turing-complete [221], practical constraints such as finite precision and limited computation time render them equivalent to finite-state automata (FSA) [222], [223]. These insights potentially bridge the gap between the intricate black-box nature of RNNs and the well-understood FSAs, which have been rigorously examined in classical formal theory.

Interestingly, attempts to leverage FSAs for RNN analysis predate these theoretical explorations, originating as early as the 1990s. These studies try to first abstract the concrete (hidden) state space and then build FSAs that try to mirror RNNs behavior. Omlin et al. introduce a method to segment the hidden state space into q equal intervals, with q being the quantization level [119]. Zeng et al. [120] and Cechin et al. [121] propose to use K-means to cluster concrete states into abstract states. These pioneering efforts from the pre-deep learning era paved the way for subsequent model-based analysis of more sophisticated RNNs.

The advent of deep learning has ushered in two transformative shifts in the field: an influx of data and increasingly complex architectures. Concurrently, the model-based analysis has also evolved accordingly. These efforts broadly fall into two categories: those that focus on extracting a transparent surrogate model replicating RNN decisions [56], [60], [123], [224], [225], [226], [227], [228], [229] and those emphasizing transition traces with associated semantic meanings related to downstream tasks [29], [54], [122], [127], [230].

For the former, one line of research leverages a more formal way for the FSA extraction, such as using Angluin’s L^* algorithm [224] and its variant [225] or finding the Hankel matrix of a black-box system and constructing weighted automata from it [226]. These strategies treat RNNs as teachers and craft automata through querying. Alternatively, a more

empirical path focuses on analyzing direct transition traces derived from training data. For example, Dong et al. first obtain symbolic states by clustering on concrete hidden states and build probabilistic automata based on a learning algorithm [60]. Zhang et al. use similar methods to build symbolic states but enhance the context-awareness of the extracted model by compositing adjacent states [123]. Merrill et al. introduce an automata extraction technique based on state merging, which performs better than k-means [228]. Hong et al. utilize a transition path matching method, integrate identified patterns with state merging, and offer a more systematic approach to constructing automata [229]. All these methods aim to extract automata for better consistency with source RNNs.

Rather than creating an exact FSA mirroring a target DNN's behavior, stakeholders may prioritize specific properties of stateful software systems, such as security, safety, privacy, and correctness. Consequently, some studies focus more on studying these specific properties and obtaining insights observed from the extracted FSA instead of seeking a perfect decision alignment. For instance, DeepStellar [54] and its successor, Marble [122], delve into the adversarial robustness of RNNs using discrete probabilistic models. Conversely, AbASG [230] employs automata for adversarial sequence generation. DeepMemory [127] performs analysis of RNN memorization and its associated security and privacy implications using semantic Markov models. RNNRepair [29] performs the repair of an RNN through model-based analysis and guidance. DeepSeer [231] employs finite automata as the central methodology for human interactive design to enable RNN debugging. The diverse successes of these methods underscore the efficacy of model-based analysis in stateful DNN systems.

Our work differentiates from the above studies in two key aspects. Firstly, we endeavor to develop a universal analysis framework designed for versatile property analysis across a broad spectrum of tasks in stateful DNN software systems in a plug-and-play manner. Secondly, our emphasis lies on the Transformer architecture and the corresponding LLMs. These models operate on a very distinct mechanism (e.g. attention mechanism) and adhere to unique training workflows. Recent studies find that the Transformer has much better empirical representation power than LSTM in simulating pushdown automation, calling the need for adapted analysis methods [232]. On the other hand, various papers have pointed out that some important capabilities of the Transformer, including factual associations [233] and object identification [234], stem from propagating complex information through tokens, inherently exhibiting stateful characteristics. While some related studies have investigated the potential of enhancing language models with finite automata for improved performance [235] or constraining their outputs using DFA [236], a comprehensive model-based analysis and framework remain to be absent.

C. LLM and Software Engineering

Recently, a growing number of research studies show that LLMs have already made great potential in various phases throughout the software production lifecycle. Many researchers and industrial practitioners have investigated and examined

the capabilities of LLMs for a large spectrum of applications in software engineering domain, such as code generation [1], [237], [238], [239], [240], code summarization [241], [242], [243], program synthesis [164], [244], test case generation [245], [246], [247], [248] and bug fixing [249], [250], [251], [252], [253].

In particular, Dong et al. [240] leverage ChapGPT to present a self-collaboration framework for code generation. Namely, multiple LLMs are assigned with different roles (i.e., coder, tester, etc.) following a general software development schema. Such an LLM-powered self-collaboration framework achieves state-of-the-art performance in solving complex real-world code generation tasks. Ahmed et al. [241] investigate the effectiveness of few-shot training on LLM (Codex [163]) for code summarization tasks. Their experiment results confirm that leveraging data from the same project with few-shot training is a promising approach to improve the performance of the LLM in code summarization. Nijkamp et al. [237] release a family of LLMs (CODEGEN) trained on both natural language and programming language data to demonstrate the ability of LLMs on program synthesis. In addition, Lemieux et al. [246] incorporate LLM into the loop to improve search-based software testing (SBST) for programs being tested through a combination of test case generation and other techniques. Last but not least, Sobania et al. [249] study the capability of ChatGPT in terms of software bug localization and fixing. These works qualify the potential of LLMs as an enabler and a booster to accelerate the software production lifecycle.

Despite the promising SE task-handling capabilities by LLMs, existing works [9], [63], [66], [160], [254], [255] have also pointed out that the current LLMs could potentially suffer critical quality issues across different SE tasks. Specifically, developers sometimes find it hard to understand the code generation process and the code produced by LLMs, and LLMs have also exhibited incorrect behaviors in generating suboptimal or erroneous solutions [66]. Such concerns about the trustworthiness of LLMs and the quality of the corresponding outcomes greatly hinder further adaptation and deployment of LLMs on safety, reliability, security, and privacy-related SE applications. Moreover, although a large body of current works in the SE community focuses on leveraging LLM to further promote and accelerate SE applications from different aspects, less attention has been paid to applying and adapting existing SE methodologies to safeguard the trustworthiness of LLMs. As the trend of LLM-based techniques for various key stages of the SE lifecycle is fast increasing, it is recognized that LLM will potentially play a more and more important role in the next few years. Therefore, it could be of great importance to establish an early foundation towards a more systematic analysis of LLMs to better interpret their behavior, to understand the potential risks when using it, and to equip the researchers and developers with more tangible guidance (e.g., concrete analysis results and feedback) to facilitate the continuous enhancement of LLMs for practical usage. Therefore, to bridge this gap and inspire further research along this direction, we hope LUNA, as a basic analysis framework for LLMs, could be helpful for researchers and practitioners to conduct more deep exploration

and exploitation and to design novel quality assurance solutions towards approaching trustworthy LLMs in practice.

VIII. CONCLUSION

In this paper, we propose *LUNA*, a model-based LLM-oriented analysis framework, to initiate an early exploration towards establishing the foundation for the trustworthiness assurance of LLMs. *LUNA* is designed to be general and extensible, and the core of its current version contains three state abstraction techniques, two model construction approaches, and as many as twelve quality metrics (as indicators) to establish a versatile LLM analysis pipeline. A large-scale evaluation of three trustworthiness perspectives on three datasets with a total of 180 model configuration settings is conducted to investigate the effectiveness of our framework. Our evaluation also performs large-scale comparative studies to better understand the strengths and weaknesses of different modeling approaches in terms of characterizing the internal behavior patterns of LLMs. Overall, advanced LLM-specific modeling methods are often needed to effectively and efficiently transparentize and interpret the behavior characteristics of LLM regardless of types of tasks and trustworthiness perspectives. Moreover, our analysis of twelve metrics motivates further investigation of strategically collaborating different metrics to provide a relatively comprehensive and meticulous quality measurement for diverse LLM applications. With the fast-growing trend of industrial adoption of LLMs across domains, we hope this early-stage exploratory work can inspire further research in this direction towards addressing many challenges to approaching trustworthy LLMs in the coming era of AI.

REFERENCES

- [1] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Proc. Chi Conf. Human Factors Comput. Syst. Extended Abstr.*, 2022, pp. 1–7.
- [2] C. S. Xia and L. Zhang, "Conversational automated program repair," 2023, *arXiv:2301.13246*.
- [3] W. Zhang, Y. Deng, B. Liu, S. J. Pan, and L. Bing, "Sentiment analysis in the era of large language models: A reality check," 2023, *arXiv:2305.15005*.
- [4] Y. Zhou et al., "Large language models are human-level prompt engineers," 2022, *arXiv:2211.01910*.
- [5] "ChatGPT." ChatGPT. Accessed: 2022. [Online]. Available: <http://chat.openai.com>
- [6] "GPT4." Accessed: 2023. <https://openai.com/gpt-4>
- [7] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [8] S. Bubeck et al., "Sparks of artificial general intelligence: Early experiments with GPT-4," 2023, *arXiv:2303.12712*.
- [9] B. Wang et al., "DecodingTrust: A comprehensive assessment of trustworthiness in GPT models," 2023, *arXiv:2306.11698*.
- [10] H. Raj, D. Rosati, and S. Majumdar, "Measuring reliability of large language models through semantic consistency," 2022, *arXiv:2211.05853*.
- [11] B. Wang et al., "InfoBERT: Improving robustness of language models from an information theoretic perspective," 2020, *arXiv:2010.02329*.
- [12] B. Alkhamissi, M. Li, A. Celikyilmaz, M. Diab, and M. Ghazvininejad, "A review on language models as knowledge bases," 2022, *arXiv:2204.06031*.
- [13] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, Mar. 2023, doi: 10.1145/3571730.
- [14] A. Abid, M. Farooqi, and J. Zou, "Persistent anti-Muslim bias in large language models," in *Proc. AAAI/ACM Conf. AI, Ethics, Soc.*, 2021, pp. 298–306.
- [15] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," 2020, *arXiv:2005.00661*.
- [16] M. Ahn et al., "Do as i can, not as i say: Grounding language in robotic affordances," 2022, *arXiv:2204.01691*.
- [17] S. Wang, Z. Zhao, X. Ouyang, Q. Wang, and D. Shen, "ChatCAD: Interactive computer-aided diagnosis on medical image using large language models," 2023, *arXiv:2302.07257p*.
- [18] K. Pei, Y. Cao, J. Yang, and S. Jana, "DeepXplore: Automated white-box testing of deep learning systems," in *Proc. 26th Symp. Operating Syst. Princ.*, 2017, pp. 1–18.
- [19] L. Ma et al., "DeepGauge: Multi-granularity testing criteria for deep learning systems," in *Proc. 33rd ACM/IEEE Int. Conf. Automated Softw. Eng.*, 2018, pp. 120–131.
- [20] J. Kim, R. Feldt, and S. Yoo, "Guiding deep learning system testing using surprise adequacy," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 1039–1049.
- [21] X. Xie et al., "DeepHunter: A coverage-guided fuzz testing framework for deep neural networks," in *Proc. 28th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2019, pp. 146–157.
- [22] L. Ma et al., "DeepMutation: Mutation testing of deep learning systems," in *Proc. IEEE 29th Int. Symp. Softw. Rel. Eng. (ISSRE)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 100–111.
- [23] Y. Tian, K. Pei, S. Jana, and B. Ray, "DeepTest: Automated testing of deep-neural-network-driven autonomous cars," in *Proc. 40th Int. Conf. Softw. Eng.*, 2018, pp. 303–314.
- [24] M. Zhang, Y. Zhang, L. Zhang, C. Liu, and S. Khurshid, "DeepRoad: GAN-based metamorphic testing and input validation framework for autonomous driving systems," in *Proc. 33rd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 132–142.
- [25] H. Wang, B. Ustun, and F. Calmon, "Repairing without retraining: Avoiding disparate impact with counterfactual distributions," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2019, pp. 6618–6627.
- [26] M. Sotoudeh and A. V. Thakur, "Correcting deep neural networks with small, generalizing patches," in *Proc. Workshop Saf. Robustness Decis. Making*, 2019, pp. 1–13.
- [27] H. Zhang and W. Chan, "Apricot: A weight-adaptation approach to fixing deep learning models," in *Proc. 34th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 376–387.
- [28] B. Yu et al., "DeepRepair: Style-guided repairing for deep neural networks in the real-world operational environment," *IEEE Trans. Rel.*, vol. 71, no. 4, pp. 1401–1416, Dec. 2022.
- [29] X. Xie et al., "RNNRepair: Automatic RNN repair via model-based analysis," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 11383–11392.
- [30] Q. Hu et al., "An empirical study on data distribution-aware test selection for deep learning enhancement," *ACM Trans. Softw. Eng. Methodol. (TOSEM)*, vol. 31, no. 4, pp. 1–30, 2022.
- [31] X. Gao, Y. Feng, Y. Yin, Z. Liu, Z. Chen, and B. Xu, "Adaptive test selection for deep neural networks," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 73–85.
- [32] Z. Yang, J. Shi, M. H. Asyofi, and D. Lo, "Revisiting neuron coverage metrics and quality of deep neural networks," in *Proc. IEEE Int. Conf. Softw. Anal., Evolution Reeng. (SANER)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 408–419.
- [33] V. Riccio and P. Tonella, "When and why test generators for deep learning produce invalid inputs: An empirical study," in *2023 IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 1161–1173.
- [34] J. Wang et al., "BET: Black-box efficient testing for convolutional neural networks," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2022, pp. 164–175.
- [35] J.-t. Huang et al., "AEON: A method for automatic evaluation of Nlp test cases," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2022, pp. 202–214.
- [36] Z. Wei, H. Wang, I. Ashraf, and W.-K. Chan, "DeepPatch: Maintaining deep learning model programs to retain standard accuracy with substantial robustness improvement," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 6, pp. 1–49, 2023.
- [37] R. Schumi and J. Sun, "Semantic-based neural network repair," 2023, *arXiv:2306.07995*.
- [38] Y. Zhang, Z. Wang, J. Jiang, H. You, and J. Chen, "Toward improving the robustness of deep learning models via model transformation," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–13.

- [39] Y. Li, M. Chen, and Q. Xu, "HybridRepair: Towards annotation-efficient repair for deep learning models," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2022, pp. 227–238.
- [40] T. Zohdinasab, V. Riccio, and P. Tonella, "DeepAtash: Focused test generation for deep learning systems," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, Jul. 2023, pp. 954–966.
- [41] T. Zohdinasab, V. Riccio, A. Gambi, and P. Tonella, "Efficient and effective feature space exploration for testing deep learning systems," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 2, pp. 1–38, Mar. 2023, doi: 10.1145/3544792.
- [42] N. Humbatova, G. Jahangirova, and P. Tonella, "DeepCrime: From real faults to mutation testing tool for deep learning," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.: Companion Proc. (ICSE-Companion)*, 2023, pp. 68–72.
- [43] A. Stocco, P. J. Nunes, M. D'Amorim, and P. Tonella, "ThirdEye: Attention maps for safe autonomous driving systems," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–12.
- [44] J. Kim, G. An, R. Feldt, and S. Yoo, "Learning test-mutant relationship for accurate fault localisation," *Inf. Softw. Technol.*, vol. 162, 2023, Art. no. 107272.
- [45] J. Sohn, S. Kang, and S. Yoo, "Arachne: Search-based repair of deep neural networks," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, pp. 1–26, 2023.
- [46] J. Kim, N. Humbatova, G. Jahangirova, P. Tonella, and S. Yoo, "Repairing DNN architecture: Are we there yet?" in *Proc. IEEE Conf. Softw. Testing, Verification Validation (ICST)*, 2023, pp. 234–245.
- [47] A. Stocco, M. Weiss, M. Calzana, and P. Tonella, "Misbehaviour prediction for autonomous driving systems," in *Proc. ACM/IEEE 42nd Int. Conf. Softw. Eng.*, 2020, pp. 359–371.
- [48] N. Humbatova, G. Jahangirova, and P. Tonella, "DeepCrime: Mutation testing of deep learning systems based on real faults," in *Proc. 30th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2021, pp. 67–78.
- [49] J. Zhou, F. Li, J. Dong, H. Zhang, and D. Hao, "Cost-effective testing of a deep learning model through input reduction," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2020, pp. 289–300.
- [50] Y. LeCun et al., "Handwritten digit recognition with a back-propagation network," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 2, 1989, pp. 396–404.
- [51] D. E. Rumelhart et al., "Learning internal representations by error propagation," in *Parallel Distributed Processing: Explorations in the Microstructure of Cognition: Foundations*, Cambridge, MA, USA: MIT Press, 1987, pp. 318–362.
- [52] T. Zohdinasab, V. Riccio, A. Gambi, and P. Tonella, "DeepHyperion: Exploring the feature space of deep learning-based systems through illumination search," in *Proc. 30th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2021, pp. 79–90.
- [53] B. C. Hu, L. Marsoo, K. Czarniecki, and M. Chechik, "What to check: Systematic selection of transformations for analyzing reliability of machine vision components," in *Proc. IEEE 33rd Int. Symp. Softw. Rel. Eng. (ISSRE)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 49–60.
- [54] X. Du, X. Xie, Y. Li, L. Ma, Y. Liu, and J. Zhao, "DeepStellar: Model-based quantitative analysis of stateful deep learning systems," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2019, pp. 477–487.
- [55] X. Ren et al., "DeepArc: Modularizing neural networks for the model maintenance," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 1008–1019.
- [56] I. Khmelnitsky et al., "Property-directed verification and robustness certification of recurrent neural networks," in *Proc. 19th Int. Symp. Automated Technol. Verification Anal. (ATVA)*, Gold Coast, QLD, Australia. New York, NY, USA: Springer-Verlag, 2021, pp. 364–380.
- [57] J. Song, X. Xie, and L. Ma, "SIEGE: A semantics-guided safety enhancement framework for AI-enabled cyber-physical systems," *IEEE Trans. Softw. Eng.*, vol. 49, no. 8, pp. 4058–4080, Aug. 2023.
- [58] X. Xie, J. Song, Z. Zhou, F. Zhang, and L. Ma, "Mosaic: Model-based safety analysis framework for AI-enabled cyber-physical systems," 2023, *arXiv:2305.03882*.
- [59] R. Pan and H. Rajan, "Decomposing convolutional neural networks into reusable and replaceable modules," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 524–535.
- [60] G. Dong et al., "Towards interpreting recurrent neural networks through probabilistic abstraction," in *Proc. 35th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2020, pp. 499–510.
- [61] H. Q et al., "ArchRepair: Block-level architecture-oriented repairing for deep neural networks," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 5, pp. 1–31, 2023.
- [62] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.
- [63] X. Hou et al., "Large language models for software engineering: A systematic literature review," 2024, *arXiv:2308.10620*.
- [64] Y. Charalambous et al., "A new era in software security: Towards self-healing software via large language models and formal verification," 2023, *arXiv:2305.14752*.
- [65] D. Lo, "Trustworthy and synergistic artificial intelligence for software engineering: Vision and roadmaps," 2023, *arXiv:2309.04142*.
- [66] A. Fan et al., "Large language models for software engineering: Survey and open problems," 2023, *arXiv:2310.03533*.
- [67] R. Luo et al., "BioGPT: generative pre-trained transformer for biomedical text generation and mining," *Briefings Bioinf.*, vol. 23, no. 69, 2022, Art. no. bbac40.
- [68] R. Taylor et al., "Galactica: A large language model for science," 2022, *arXiv:2211.09085*.
- [69] Y. Shen et al., "ChatGPT and other large language models are double-edged swords," *Radiology*, vol. 307, no. 2, 2023, Art. no. e230163.
- [70] T. Kocmi and C. Federmann, "Large language models are state-of-the-art evaluators of translation quality," 2023, *arXiv:2302.14520*.
- [71] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individual Differences*, vol. 103, 2023, Art. no. 102274.
- [72] D. B. Lenat, "Cyc: A large-scale investment in knowledge infrastructure," *Commun. ACM*, vol. 38, no. 11, p. 33–38, Nov. 1995, doi: 10.1145/219717.219745.
- [73] Y. Liu et al., "Summary of ChatGPT/GPT-4 research and perspective towards the future of large language models," 2023, *arXiv:2304.01852*.
- [74] R. Mao, Q. Liu, K. He, W. Li, and E. Cambria, "The biases of pre-trained language models: An empirical study on prompt-based sentiment analysis and emotion detection," *IEEE Trans. Affect. Comput.*, vol. 14, no. 3, pp. 1743–1753, Jul.–Sep. 2023.
- [75] T. Zhang, F. Ladhak, E. Durmus, P. Liang, K. McKeown, and T. B. Hashimoto, "Benchmarking large language models for news summarization," 2023, *arXiv:2301.13848*.
- [76] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *Proc. 6th ACM SIGPLAN Int. Symp. Mach. Program.*, 2022, pp. 1–10.
- [77] T. Ahmed and P. Devanbu, "Few-shot training llms for project-specific code-summarization," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–5.
- [78] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, Mar. 1994.
- [79] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2013, pp. 1310–1318.
- [80] I. Malkiel, D. Ginzburg, O. Barkan, A. Caciularu, J. Weill, and N. Koenigstein, "Interpreting BERT-based text similarity via activation and saliency maps," in *Proc. ACM Web Conf.*, 2022, pp. 3259–3268.
- [81] A. Azaria and T. Mitchell, "The internal state of an LLM knows when its lying," 2023, *arXiv:2304.13734*.
- [82] H. Chefer, S. Gur, and L. Wolf, "Generic attention-model explainability for interpreting bi-modal and encoder-decoder transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 397–406.
- [83] X. Li et al., "Interpretable deep learning: Interpretation, interpretability, trustworthiness, and beyond," *Knowl. Inf. Syst.*, vol. 64, no. 12, pp. 3197–3234, 2022.
- [84] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (Long and Short Papers)*, vol. 1. Minneapolis, MN, USA: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423>
- [85] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," 2020, *arXiv:2006.03654*.
- [86] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
- [87] M. Lewis et al., "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*. Online: Association for Computational Linguistics, Jul. 2020, pp. 7871–7880. [Online]. Available: <https://aclanthology.org/2020.acl-main.703>
- [88] Y. Tay et al., "UL2: Unifying language learning paradigms," in *Proc. 11th Int. Conf. Learn. Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=6ruVLB727MC>

- [89] C. Raffel et al., “Exploring the limits of transfer learning with a unified text-to-text transformer,” *J. Mach. Learn. Res.*, vol. 21, no. 1, pp. 5485–5551, 2020.
- [90] T. Brown et al., “Language models are few-shot learners,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
- [91] B. Roziere et al., “Code Llama: Open foundation models for code,” 2023, *arXiv:2308.12950*.
- [92] M. T. Ribeiro, S. Singh, and C. Guestrin, ““Why should i trust you?” Explaining the predictions of any classifier,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 1135–1144.
- [93] H. Qiu, S. Zhang, A. Li, H. He, and Z. Lan, “Latent Jailbreak: A benchmark for evaluating text safety and output robustness of large language models,” 2023, *arXiv:2307.08487*.
- [94] Y. Li, F. Wei, J. Zhao, C. Zhang, and H. Zhang, “Rain: Your language models can align themselves without finetuning,” 2023, *arXiv:2309.07124*.
- [95] A. Helbling, M. Phute, M. Hull, and D. H. Chau, “LLM self defense: By self examination, llms know they are being tricked,” 2023, *arXiv:2308.07308*.
- [96] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” 2016, *arXiv:1606.06565*.
- [97] D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–12. [Online]. Available: <https://openreview.net/forum?id=Hkg4TI9xl>
- [98] K. Lee, K. Lee, H. Lee, and J. Shin, “A simple unified framework for detecting out-of-distribution samples and adversarial attacks,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018.
- [99] S. Liang, Y. Li, and R. Srikant, “Enhancing the reliability of out-of-distribution image detection in neural networks,” in *Proc. Int. Conf. Learn. Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=H1VGkIXRZ>
- [100] P. Morteza and Y. Li, “Provable guarantees for understanding out-of-distribution detection,” in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 7, pp. 7831–7840.
- [101] H. Lang, Y. Zheng, Y. Li, J. Sun, F. Huang, and Y. Li, “A survey on out-of-distribution detection in NLP,” 2023, *arXiv:2305.03236*.
- [102] U. Arora, W. Huang, and H. He, “Types of out-of-distribution texts and how to detect them,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 10687–10701. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.835>
- [103] J. Ren et al., “Out-of-distribution detection and selective generation for conditional language models,” in *Proc. 11th Int. Conf. Learn. Representations*, 2023, pp. 1–32. [Online]. Available: <https://openreview.net/forum?id=kJUSnD0vPB>
- [104] A. Kamath, R. Jia, and P. Liang, “Selective question answering under domain shift,” in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*. Online: Association for Computational Linguistics, Jul. 2020, pp. 5684–5696. [Online]. Available: <https://aclanthology.org/2020.acl-main.503>
- [105] J. Wang et al., “On the robustness of ChatGPT: An adversarial and out-of-distribution perspective,” 2023, *arXiv:2302.12095*.
- [106] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera, “A unifying view on dataset shift in classification,” *Pattern Recognit.*, vol. 45, no. 1, pp. 521–530, 2012.
- [107] B. Biggio et al., “Evasion attacks against machine learning at test time,” in *Proc. Eur. Conf. Mach. Learn. Knowl. Discovery Databases (ECML PKDD)*, Prague, Czech Republic (Part III 13). New York, NY, USA: Springer-Verlag, 2013, pp. 387–402.
- [108] C. Szegedy et al., “Intriguing properties of neural networks,” in *Proc. 2nd Int. Conf. Learn. Representations (ICLR)*, 2014, pp. 1–10.
- [109] I. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” in *Proc. Int. Conf. Learn. Representations*, 2015, *arXiv:1412.6572*.
- [110] A. Chakraborty, M. Alam, V. Dey, A. Chattopadhyay, and D. Mukhopadhyay, “Adversarial attacks and defences: A survey,” 2018, *arXiv:1810.00069*.
- [111] B. Wang et al., “Adversarial glue: A multi-task benchmark for robustness evaluation of language models,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 1–24.
- [112] S. Goyal, S. Doddapaneni, M. M. Khapra, and B. Ravindran, “A survey of adversarial defenses and robustness in NLP,” *ACM Comput. Surveys*, vol. 55, no. 14s, pp. 1–39, 2023.
- [113] V. Raunak, A. Menezes, and M. Junczys-Dowmunt, “The curious case of hallucinations in neural machine translation,” in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, 2021, pp. 1172–1183.
- [114] A. Rohrbach, L. A. Hendricks, K. Burns, T. Darrell, and K. Saenko, “Object hallucination in image captioning,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2018, pp. 4035–4045.
- [115] P. Koehn and R. Knowles, “Six challenges for neural machine translation,” 2017, *arXiv:1706.03872*.
- [116] W. Kryściński, B. McCann, C. Xiong, and R. Socher, “Evaluating the factual consistency of abstractive text summarization,” 2019, *arXiv:1910.12840*.
- [117] B. Bi, C. Wu, M. Yan, W. Wang, J. Xia, and C. Li, “Incorporating external knowledge into machine reading for generative question answering,” 2019, *arXiv:1909.02745*.
- [118] A. Balakrishnan, J. Rao, K. Upasani, M. White, and R. Subba, “Constrained decoding for neural NLG from compositional representations in task-oriented dialogue,” in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*. Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 831–844. [Online]. Available: <https://www.aclweb.org/anthology/P19-1080>
- [119] C. W. Omlin and C. L. Giles, “Extraction of rules from discrete-time recurrent neural networks,” *Neural Netw.*, vol. 9, no. 1, pp. 41–52, 1996.
- [120] Z. Zeng, R. M. Goodman, and P. Smyth, “Learning finite state machines with self-clustering recurrent networks,” *Neural Comput.*, vol. 5, no. 6, pp. 976–990, 1993.
- [121] A. L. Cechin, D. Regina, P. Simon, and K. Stertz, “State automata extraction from recurrent neural nets using k-means and fuzzy clustering,” in *Proc. 23rd Int. Conf. Chilean Comput. Sci. Soc. (SCCC)*, Piscataway, NJ, USA: IEEE Press, 2003, pp. 73–78.
- [122] X. Du, Y. Li, X. Xie, L. Ma, Y. Liu, and J. Zhao, “Marble: Model-based robustness analysis of stateful deep learning systems,” in *Proc. 35th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2020, pp. 423–435.
- [123] X. Zhang, X. Du, X. Xie, L. Ma, Y. Liu, and M. Sun, “Decision-guided weighted automata extraction from recurrent neural networks,” in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 13, pp. 11699–11707.
- [124] Y. Ming et al., “Understanding hidden memories of recurrent neural networks,” in *Proc. IEEE Conf. Vis. Analytics Sci. Technol. (VAST)*, Piscataway, NJ, USA: IEEE Press, 2017, pp. 13–24.
- [125] “Countable-state Markov chains.” MIT OpenCourseWare. Accessed: 2011. [Online]. Available: https://ocw.mit.edu/courses/6-262-discrete-stochastic-processes-spring-2011/01d0892549619cb25d928f15ec7230ed_MIT6_262S11_chap05.pdf
- [126] M. Fan, Z. Si, X. Xie, Y. Liu, and T. Liu, “Text backdoor detection using an interpretable RNN abstract model,” *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4117–4132, 2021.
- [127] D. Zhu, J. Chen, W. Shang, X. Zhou, J. Grossklags, and A. E. Hassan, “DeepMemory: Model-based memorization analysis of deep neural language models,” in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 1003–1015.
- [128] B. Sun, J. Sun, L. H. Pham, and J. Shi, “Causality-based neural network repair,” in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 338–349.
- [129] B. Sun, J. Sun, T. Dai, and L. Zhang, “Probabilistic verification of neural networks against group fairness,” in *Proc. 24th Int. Symp. Formal Methods (FM)*, Virtual Event. New York, NY, USA: Springer-Verlag, 2021, pp. 83–102.
- [130] Z. Wei, X. Zhang, and M. Sun, “Extracting weighted finite automata from recurrent neural networks for natural languages,” in *Proc. Int. Conf. Formal Eng. Methods*, New York, NY, USA: Springer-Verlag, 2022, pp. 370–385.
- [131] R. Bro and A. K. Smilde, “Principal component analysis,” *Analytical Methods*, vol. 6, no. 9, pp. 2812–2831, 2014.
- [132] D. A. Reynolds et al., “Gaussian mixture models,” in *Encyclopedia of Biometrics*, Berlin, Germany: Springer, vol. 741, pp. 659–663, 2009.
- [133] K. Krishna and M. N. Murty, “Genetic k-means algorithm,” *IEEE Trans. Syst., Man, Cybern., Part B (Cybern.)*, vol. 29, no. 3, pp. 433–439, Jun. 1999.
- [134] S. R. Eddy, “Profile hidden Markov models,” *Bioinformatics* (Oxford, England), vol. 14, no. 9, pp. 755–763, 1998.
- [135] L. Rabiner and B. Juang, “An introduction to hidden Markov models,” *IEEE ASSP Mag.*, vol. 3, no. 1, pp. 4–16, Jan. 1986.
- [136] S. Fine, Y. Singer, and N. Tishby, “The hierarchical hidden Markov model: Analysis and applications,” *Mach. Learn.*, vol. 32, pp. 41–62, Jul. 1998.
- [137] L. E. Baum, T. Petrie, G. Soules, and N. Weiss, “A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains,” *Ann. Math. Statist.*, vol. 41, no. 1, pp. 164–171, 1970.

- [138] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," 2021, *arXiv:2109.07958*.
- [139] E. Ozen and A. Orailoglu, "Sanity-check: Boosting the reliability of safety-critical deep neural network applications," in *Proc. IEEE 28th Asian Test Symp. (ATS)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 7–75.
- [140] O. Kupferman, "Sanity checks in formal verification," in *Proc. Int. Conf. Concurrency Theory*, New York, NY, USA: Springer-Verlag, 2006, pp. 37–51.
- [141] Y. Ishimoto, M. Kondo, N. Ubayashi, and Y. Kamei, "PAFL: Probabilistic automaton-based fault localization for recurrent neural networks," *Inf. Softw. Technol.*, vol. 155, 2023, Art. no. 107117.
- [142] B. G. Vegetabile, S. A. Stout-Oswald, E. P. Davis, T. Z. Baram, and H. S. Stern, "Estimating the entropy rate of finite Markov chains with application to behavior studies," *J. Educ. Behav. Statist.*, vol. 44, no. 3, pp. 282–308, 2019.
- [143] E. Nummelin, *General Irreducible Markov Chains and Non-Negative Operators*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [144] N. Carlini et al., "Extracting training data from large language models," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur. 21)*, 2021, pp. 2633–2650.
- [145] R. Matinnejad, S. Nejati, L. C. Briand, and T. Bruckmann, "Test generation and test prioritization for simulink models with dynamic behavior," *IEEE Trans. Softw. Eng.*, vol. 45, no. 9, pp. 919–944, Sep. 2019.
- [146] G. Gigerenzer and U. Hoffrage, "How to improve Bayesian reasoning without instruction: Frequency formats," *Psychol. Rev.*, vol. 102, no. 4, 1995, Art. no. 684.
- [147] P. Manakul, A. Liusie, and M. J. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," 2023, *arXiv:2303.08896*.
- [148] Y. Chang et al., "A survey on evaluation of large language models," 2023, *arXiv:2307.03109*.
- [149] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 27730–27744.
- [150] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in *Proc. ACM Conf. Fairness, Accountability, Transparency*, 2021, pp. 610–623.
- [151] Z. Kenton, T. Everitt, L. Weidinger, I. Gabriel, V. Mikulik, and G. Irving, "Alignment of language agents," 2021, *arXiv:2103.14659*.
- [152] S. Santhanam et al., "Rome was built in 1776: A case study on factual correctness in knowledge-grounded response generation," 2021, *arXiv:2110.05456*.
- [153] Y. Huang, X. Feng, X. Feng, and B. Qin, "The factual inconsistency problem in abstractive text summarization: A survey," 2021, *arXiv:2104.14839*.
- [154] A. P. Bradley, "The use of the area under the ROC curve in the evaluation of machine learning algorithms," *Pattern Recognit.*, vol. 30, no. 7, pp. 1145–1159, 1997.
- [155] Y. Wang et al., "Self-instruct: Aligning language models with self-generated instructions," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*. Dublin, Ireland: Association for Computational Linguistics, 2023, pp. 13484–13508.
- [156] W.-L. Chiang et al., "Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality," LMSYS Org. Accessed: 2023. [Online]. Available: <https://lmsys.org/blog/2023-03-30-vicuna/>
- [157] R. Taori et al., "Stanford alpaca: An instruction-following LLaMA model," Github. Accessed: 2023. [Online]. Available: https://github.com/tatsu-lab/stanford_alpaca
- [158] H. Touvron et al., "LLaMA 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [159] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," 2023, *arXiv:2305.01210*.
- [160] X. Huang et al., "A survey of safety and trustworthiness of large language models through the lens of verification and validation," 2023, *arXiv:2305.11391*.
- [161] T. Liu et al., "A token-level reference-free hallucination detection benchmark for free-form text generation," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 6723–6737. [Online]. Available: <https://aclanthology.org/2022.acl-long.464>
- [162] R. Socher et al., "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2013, pp. 1631–1642.
- [163] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.
- [164] J. Austin et al., "Program synthesis with large language models," 2021, *arXiv:2108.07732*.
- [165] X. Hu, G. Li, X. Xia, D. Lo, S. Lu, and Z. Jin, "Summarizing source code with transferred API knowledge," 2018.
- [166] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," 2019, *arXiv:1909.09436*.
- [167] E. Shi et al., "On the evaluation of neural code summarization," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1597–1608.
- [168] "GPT 3.5." OpenAI. Accessed: 2022. [Online]. Available: <https://platform.openai.com/docs/models/gpt-3-5>
- [169] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg, "Inference-time intervention: Eliciting truthful answers from a language model," 2023, *arXiv:2306.03341*.
- [170] G. R. Terrell and D. W. Scott, "Variable kernel density estimation," *Ann. Statist.*, vol. 45, no. 3, pp. 1236–1265, 1992.
- [171] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *Ann. Math. Statist.*, vol. 18, no. 1, pp. 50–60, 1947.
- [172] I. Cohen et al., "Pearson correlation coefficient," in *Noise Reduction in Speech Processing*. Berlin, Germany: Springer, pp. 1–4, 2009.
- [173] M. G. Kendall, "A new measure of rank correlation," *Biometrika*, vol. 30, no. 1/2, pp. 81–93, 1938.
- [174] P. Y. Chen and P. M. Popovich, *Correlation: Parametric and Nonparametric Measures*. Newbury Park, CA, USA: Sage, 2002.
- [175] S. J. Semnani, V. Z. Yao, H. C. Zhang, and M. S. Lam, "WikiChat: A few-shot LLM-based chatbot grounded with Wikipedia," 2023, *arXiv:2305.14292*.
- [176] S. Chorev et al., "DeepChecks: A library for testing and validating machine learning models and data," *J. Mach. Learn. Res.*, vol. 23, no. 285, pp. 1–6, 2022. [Online]. Available: <http://jmlr.org/papers/v23/22-0281.html>
- [177] C.-H. Cheng, G. Nührenberg, and H. Yasuoka, "Runtime monitoring neuron activation patterns," in *Proc. Des., Automat. Test Europe Conf. Exhib. (DATE)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 300–303.
- [178] T. A. Henzinger, A. Lukina, and C. Schilling, "Outside the box: Abstraction-based monitoring of neural networks," 2019, *arXiv:1911.09032*.
- [179] Q. M. Rahman, P. Corke, and F. Dayoub, "Run-time monitoring of machine learning for robotic perception: A survey of emerging trends," *IEEE Access*, vol. 9, pp. 20067–20075, 2021.
- [180] M. Wardat, W. Le, and H. Rajan, "DeepLocalize: Fault localization for deep neural networks," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 251–262.
- [181] Y. Wu, Z. Li, J. M. Zhang, M. Papadakis, M. Harman, and Y. Liu, "Large language models in fault localisation," 2023, *arXiv:2308.15276*.
- [182] T.-W. Weng et al., "Evaluating the robustness of neural networks: An extreme value theory approach," 2018, *arXiv:1801.10578*.
- [183] Y. Sun, X. Huang, D. Kroening, J. Sharp, M. Hill, and R. Ashmore, "Testing deep neural networks," 2018, *arXiv:1803.04792*.
- [184] D. Huang, Q. Bu, J. Zhang, X. Xie, J. Chen, and H. Cui, "Bias assessment and mitigation in llm-based code generation," 2023.
- [185] X. Song, Y. Sun, M. A. Mustafa, and L. C. Cordeiro, "AIREPAIR: A repair platform for neural networks," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.: Companion Proc. (ICSE-Companion)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 98–101.
- [186] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: A large-scale dataset for fact extraction and VERification," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (Long Papers)*, vol. 1. New Orleans, LA, USA: Association for Computational Linguistics, Jun. 2018, pp. 809–819. [Online]. Available: <https://aclanthology.org/N18-1074>
- [187] O. Honovich, L. Choshen, R. Aharoni, E. Neeman, I. Szpektor, and O. Abend, "q²: Evaluating factual consistency in knowledge-grounded dialogues via question generation and question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 7856–7870. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.619>
- [188] X. Wang et al., "TextFlint: Unified multilingual robustness evaluation toolkit for natural language processing," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics/11th Int. Joint Conf. Natural Lang. Process.: System Demonstrations*. Online: Association for Computational Linguistics, Aug. 2021, pp. 347–355. [Online]. Available: <https://aclanthology.org/2021.acl-demo.41>

- [189] S. Gehman, S. Gururangan, M. Sap, Y. Choi, and N. A. Smith, "RealToxicityPrompts: Evaluating neural toxic degeneration in language models," in *Proc. Findings Assoc. Comput. Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, Nov. 2020, pp. 3356–3369. [Online]. Available: <https://aclanthology.org/2020.findings-emnlp.301>
- [190] R. Zhao, X. Li, Y. K. Chia, B. Ding, and L. Bing, "Can ChatGPT-like generative models guarantee factual accuracy? On the mistakes of new generation search engines," 2023, *arXiv:2304.11076*.
- [191] R. Khoury, A. R. Avila, J. Brunelle, and B. M. Camara, "How secure is code generated by ChatGPT?" 2023, *arXiv:2304.09655*.
- [192] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," 2023, *arXiv:2305.01210*.
- [193] S. Frieder et al., "Mathematical capabilities of chatgpt," 2023, *arXiv:2301.13867*.
- [194] H. Liu, R. Ning, Z. Teng, J. Liu, Q. Zhou, and Y. Zhang, "Evaluating the logical reasoning ability of ChatGPT and GPT-4," 2023, *arXiv:2304.03439*.
- [195] P. Liang et al., "Holistic evaluation of language models," 2022, *arXiv:2211.09110*.
- [196] K. Filippova, "Controlled hallucinations: Learning to generate faithfully from noisy data," 2020, *arXiv:2010.05873*.
- [197] C. Zhou et al., "Detecting hallucinated content in conditional neural sequence generation," in *Proc. Findings Assoc. Comput. Linguistics (ACL-IJCNLP)*. Online: Association for Computational Linguistics, Aug. 2021, pp. 1393–1404. [Online]. Available: <https://aclanthology.org/2021.findings-acl.120>
- [198] T. Markov et al., "A holistic approach to undesired content detection in the real world," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 12, pp. 15009–15018.
- [199] T. Zhang et al., "Interpretable unified language checking," 2023, *arXiv:2304.03728*.
- [200] A. Malinin and M. Gales, "Uncertainty estimation in autoregressive structured prediction," 2020, *arXiv:2002.07650*.
- [201] Y. Xiao and W. Y. Wang, "On hallucination and predictive uncertainty in conditional language generation," in *Proc. 16th Conf. Eur. Chapter Assoc. Comput. Linguistics: Main Volume*. Online: Association for Computational Linguistics, Apr. 2021, pp. 2734–2744. [Online]. Available: <https://aclanthology.org/2021.eacl-main.236>
- [202] Y. Huang, J. Song, Z. Wang, H. Chen, and L. Ma, "Look before you leap: An exploratory study of uncertainty measurement for large language models," 2023, *arXiv:2307.10236*.
- [203] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," in *Proc. 11th Int. Conf. Learn. Representations*, 2023, pp. 1–19. [Online]. Available: <https://openreview.net/forum?id=VD-AyTP0dve>
- [204] Z. Lin, S. Trivedi, and J. Sun, "Generating with confidence: Uncertainty quantification for black-box large language models," 2023, *arXiv:2305.19187*.
- [205] J. Baan et al., "Uncertainty in natural language generation: From theory to applications," 2023, *arXiv:2307.15703*.
- [206] Z. Hu, Z. Yang, X. Liang, R. Salakhutdinov, and E. P. Xing, "Toward controlled generation of text," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2017, pp. 1587–1596.
- [207] N. Lee et al., "Factuality enhanced language models for open-ended text generation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 34586–34599.
- [208] F. Mireshghallah, K. Goyal, and T. Berg-Kirkpatrick, "Mix and match: Learning-free controllable text generation using energy language models," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 401–415. [Online]. Available: <https://aclanthology.org/2022.acl-long.31>
- [209] M. Cao, M. Fatemi, J. C. K. Cheung, and S. Shabianian, "Systematic rectification of language models via dead-end analysis," 2023, *arXiv:2302.14003*.
- [210] N. Tandon, A. Madaan, P. Clark, and Y. Yang, "Learning to repair: Repairing model output errors after deployment using a dynamic memory of feedback," in *Proc. Findings Assoc. Comput. Linguistics (NAACL)*, Seattle, WA, USA: Association for Computational Linguistics, Jul. 2022, pp. 339–352. [Online]. Available: <https://aclanthology.org/2022.findings-naacl.26>
- [211] Z. Gou et al., "Critic: Large language models can self-correct with tool-interactive critiquing," 2023, *arXiv:2305.11738*.
- [212] J. Huang et al., "Large language models can self-improve," 2022, *arXiv:2210.11610*.
- [213] A. Madaan et al., "Self-refine: Iterative refinement with self-feedback," 2023, *arXiv:2303.17651*.
- [214] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," 2023, *arXiv:2304.05128*.
- [215] H. He, H. Zhang, and D. Roth, "Rethinking with retrieval: Faithful large language model inference," 2022, *arXiv:2301.00303*.
- [216] B. Peng et al., "Check your facts and try again: Improving large language models with external knowledge and automated feedback," 2023, *arXiv:2302.12813*.
- [217] H. Qian et al., "WebBrain: Learning to generate factually correct articles for queries by grounding on large web corpus," 2023, *arXiv:2304.04358*.
- [218] W. Shi et al., "Replug: Retrieval-augmented black-box language models," 2023, *arXiv:2301.12652*.
- [219] S. Zhang, L. Pan, J. Zhao, and W. Y. Wang, "Mitigating language model hallucination with interactive question-knowledge alignment," 2023, *arXiv:2305.13669*.
- [220] Y. Ma, C. Zhang, and S.-C. Zhu, "Brain in a vat: On missing pieces towards artificial general intelligence in large language models," 2023, *arXiv:2307.03762*.
- [221] H. T. Siegelmann and E. D. Sontag, "On the computational power of neural nets," in *Proc. 5th Annu. Workshop Comput. Learn. Theory*, 1992, pp. 440–449.
- [222] G. Weiss, Y. Goldberg, and E. Yahav, "On the practical computational power of finite precision RNNs for language recognition," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics (Volume 2: Short Papers)*. Melbourne, Australia: Association for Computational Linguistics, Jul. 2018, pp. 740–745. [Online]. Available: <https://aclanthology.org/P18-2117>
- [223] W. Merrill, "Sequential neural networks as automata," in *Proc. Workshop Deep Learn. Formal Lang. Building Bridges*. Florence, Italy: Association for Computational Linguistics, Aug. 2019, pp. 1–13. [Online]. Available: <https://aclanthology.org/W19-3901>
- [224] G. Weiss, Y. Goldberg, and E. Yahav, "Extracting automata from recurrent neural networks using queries and counterexamples," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 5247–5256.
- [225] G. Weiss, Y. Goldberg, and E. Yahav, "Learning deterministic weighted automata with queries and counterexamples," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–12.
- [226] S. Ayache, R. Eyraud, and N. Goudian, "Explaining black boxes on sequential data using weighted automata," in *Proc. Int. Conf. Grammatical Inference*, PMLR, 2019, pp. 81–103.
- [227] Z. Wei, X. Zhang, Y. Zhang, and M. Sun, "Weighted automata extraction and explanation of recurrent neural networks for natural language tasks," 2023, *arXiv:2306.14040*.
- [228] W. Merrill and N. Tsilivis, "Extracting finite automata from RNNs using state merging," 2022, *arXiv:2201.12451*.
- [229] D. Hong, A. M. Segre, and T. Wang, "AdaAX: Explaining recurrent neural networks by learning automata with adaptive states," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2022, pp. 574–584.
- [230] M. Ma, D. Du, Y. Liu, Y. Wang, and Y. Li, "Efficient adversarial sequence generation for RNN with symbolic weighted finite automata," in *SafeAI@ AAAI*, 2022, pp. 1–9.
- [231] Z. Wang, Y. Huang, D. Song, L. Ma, and T. Zhang, "DeepSeer: Interactive RNN explanation and debugging via state abstraction," in *Proc. CHI Conf. Human Factors Comput. Syst.*, 2023, pp. 1–20.
- [232] H. Shi, S. Gao, Y. Tian, X. Chen, and J. Zhao, "Learning bounded context-free-grammar via LSTM and the transformer: Difference and the explanations," in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 8, pp. 8267–8276.
- [233] M. Geva, J. Bastings, K. Filippova, and A. Globerson, "Dissecting recall of factual associations in auto-regressive language models," 2023, *arXiv:2304.14767*.
- [234] K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt, "Interpretability in the wild: A circuit for indirect object identification in GPT-2 small," 2022, *arXiv:2211.00593*.
- [235] U. Alon, F. Xu, J. He, S. Sengupta, D. Roth, and G. Neubig, "Neuro-symbolic language modeling with automaton-augmented retrieval," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2022, pp. 468–485.
- [236] M. Kuchnik, V. Smith, and G. Amvrosiadis, "Validating large language models with RELM," *Proc. Mach. Learn. Syst.*, vol. 5, 2023, pp. 1–20.
- [237] E. Nijkamp et al., "CodeGen: An open large language model for code with multi-turn program synthesis," 2022, *arXiv:2203.13474*.

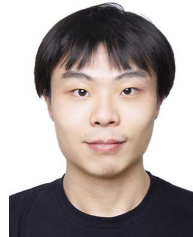
- [238] A. Svyatkovskiy, S. K. Deng, S. Fu, and N. Sundaresan, "Intellicode compose: Code generation using transformer," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2020, pp. 1433–1443.
- [239] Y. Li et al., "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [240] Y. Dong, X. Jiang, Z. Jin, and G. Li, "Self-collaboration code generation via ChatGPT," 2023, *arXiv:2304.07590*.
- [241] T. Ahmed and P. Devanbu, "Few-shot training LLMS for project-specific code-summarization," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE '22)*, New York, NY, USA: ACM, 2023, doi: 10.1145/3551349.3559555.
- [242] B. Wei, G. Li, X. Xia, Z. Fu, and Z. Jin, "Code generation as a dual task of code summarization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 6563–6573.
- [243] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," 2021, *arXiv:2109.00859*.
- [244] D. Fried et al., "InCoder: A generative model for code infilling and synthesis," 2022, *arXiv:2204.05999*.
- [245] Z. Liu et al., "Fill in the blank: Context-aware automated text input generation for mobile GUI testing," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 1355–1367.
- [246] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proc. Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 919–931.
- [247] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2024, pp. 1–17.
- [248] Y. Deng, C. S. Xia, C. Yang, S. D. Zhang, S. Yang, and L. Zhang, "Large language models are edge-case fuzzers: Testing deep learning libraries via fuzzGPT," 2023, *arXiv:2304.02014*.
- [249] D. Sobania, M. Briesch, C. Hanna, and J. Petke, "An analysis of the automatic bug fixing performance of ChatGPT," 2023, *arXiv:2301.08653*.
- [250] J. A. Prenner, H. Babii, and R. Robbes, "Can OpenAI's codex fix bugs? An evaluation on quixbugs," in *Proc. 3rd Int. Workshop Automated Program Repair*, 2022, pp. 69–75.
- [251] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 1430–1442.
- [252] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 1482–1494.
- [253] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated repair of programs from large language models," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 1469–1481.
- [254] Y. Liu et al., "Refining ChatGPT-generated code: Characterizing and mitigating code quality issues," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 5, pp. 1–26, 2023.
- [255] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language model: Survey, landscape, and vision," 2023, *arXiv:2307.07221*.



Da Song (Graduate Student Member, IEEE) received the B.S. degree from the University of Minnesota, Twin Cities, USA. He is currently working toward the Ph.D. degree with the University of Alberta, Canada. His research interests include intersection of Software Engineering (SE), AI, and human-computer interaction (HCI). He focuses on software testing and analysis for large language models, as well as designing and developing interactive interfaces that can enable more efficient human-in-the-loop AI development.



Xuan Xie received the B.E. degree from Sun Yat-sen University, China, and the master's degree from Max Planck Institute for Software Systems, Germany. He is currently working toward the Ph.D. degree with the University of Alberta, Canada. His research interests include trustworthiness analysis for large language model, safety analysis of AI-enabled cyber-physical systems, and neural network verification.



Jiayang Song (Graduate Student Member, IEEE) received the B.E. degree from the Western University, Canada, and the M.E. degree from the University of Toronto, Canada. He is currently working toward the Ph.D. degree with the University of Alberta, Canada. His research interests include testing, analysis, repairing and enhancement of AI systems and their applications to quality assurance of trustworthy AI-enabled cyber-physical systems.



Derui Zhu is currently working toward the Ph.D. degree with the Technical University of Munich. His research interests include the intersection of security, privacy, and software engineering, with a specific emphasis on trustworthy AI. He focuses particularly on vulnerability analysis and data protection within AI models.



Yuheng Huang received the B.E. degree from Beijing University of Posts and Telecommunications, China, and the master's degree from the University of Alberta, Canada. He is currently working toward the Ph.D. degree with the University of Tokyo, Japan. His research interests include the quality assurance of AI-enabled complex systems as well as developing interactive interfaces that can enable more efficient human-in-the-loop AI development from human-computer interaction (HCI) perspectives.



Felix Juefei-Xu received the Ph.D. degree in electrical and computer engineering from Carnegie Mellon University (CMU), Pittsburgh, PA, USA. Prior to that, he received the B.S. degree in electronic engineering from Shanghai Jiao Tong University (SJTU), Shanghai, China, and the M.S. degree in electrical and computer engineering and machine learning from CMU. Currently, he is a Research Scientist with GenAI at Meta, based in New York City, where he works on robust perception and efficient learning problems in the domain of generative AI. He is also affiliated with New York University as an Adjunct Professor. Previously, he was a Research Scientist with Alibaba Group, based in Sunnyvale, CA. He was the recipient of Multiple Best and Distinguished Paper Awards, including IJCB 2011, BTAS 2015 and 2016, ASE 2018, and ACCV 2018.



Lei Ma (Member, IEEE) received the B.E. degree from Shanghai Jiao Tong University, China, and the M.E. and Ph.D. degrees from the University of Tokyo, Japan. He is currently an Associate Professor with the University of Tokyo, Japan and University of Alberta, Canada. His research interests include the interdisciplinary fields of software engineering (SE) and trustworthy artificial intelligence with a special focus on the quality, reliability, safety and security aspects of AI systems. He was honorably selected as a Canada CIFAR AI Chair and Fellow at Alberta Machine Intelligence Institute (AMII).